



北方民族大学

本科毕业论文（设计）

题目：基于 Vulkan 的实时光线追踪混合渲染器实现

学院名称：计算机科学与工程学院

专业：软件工程

学生姓名：向晨

学号：20221889

指导教师姓名：韩强

企业指导教师：

论文提交时间：

北方民族大学本科毕业论文（设计）诚信承诺书

学生姓名	向晨	年级	2022 级
所学专业	软件工程	学号	20221889
所在学院	计算机科学与工程学院		

学生承诺

本人郑重承诺和声明：

我承诺在毕业论文（设计）过程中严格遵守学校有关规定，在指导教师的安排与指导下独立完成所规定的毕业论文（设计）工作，决不弄虚作假，不请别人代做毕业论文（设计）或抄袭别人的成果。所撰写的毕业论文或毕业设计是在指导老师的指导下自主完成，文中所有引文或引用数据、图表均注解并说明来源，本人愿意为由此引起的后果承担责任。

学生（签名）：_____

2026 年 05 月 日

基于 Vulkan 的实时光线追踪混合渲染器实现

摘要

在现代实时计算机图形学中，传统的纯光栅化渲染在处理全局光照、精确物理软阴影与复杂镜面反射时存在固有的物理局限性，而离线级别的纯路径追踪又难以满足交互式应用的实时帧率需求。针对这一“画质与算力”的矛盾，本文基于现代显式图形 API Vulkan 1.3，设计并实现了一套工业级的高性能实时光线追踪混合渲染引擎。

本文的核心工作包括三个方面。首先，针对 Vulkan 繁琐的显存状态同步痛点，设计了数据驱动的 Render Graph 调度架构，通过有向无环图的拓扑分析实现了管线屏障的自动推导与显存复用优化。其次，构建了高效的混合光照管线，将几何光栅化与硬件光线追踪深度解耦。在延迟渲染 G-Buffer 的基础上，结合偏导数面法线偏移与 Ray Query 扩展实现了精准无伪影的软硬阴影，并利用 RT Pipeline 全面求解 Cook-Torrance 微面元模型，完成了基于物理（PBR）的多次镜面反射计算。最后，针对 1 SPP 极低采样率带来的蒙特卡洛高频噪声，完整集成了时空方差导向滤波（SVGF）降噪系统。通过几何运动矢量的时域历史重投影与动态方差引导的多轮 A-Trous 空域滤波，成功实现了高保真度的抗噪平滑处理。

性能测试与效果分析表明，本系统在 2K 原生分辨率下，能够以实时的性能开销（稳定大于 60 帧）输出平滑且物理正确的照片级渲染画面。本文的研究验证了以 Render Graph 为底座、结合光栅化、硬件光追与 SVGF 降噪的混合架构在现代底层图形引擎开发中的高效性与工程可行性。

关键词: Vulkan; 实时光线追踪; 混合渲染; Render Graph; 基于物理的渲染 (PBR); SVGF 降噪

Implementation of a Real-time Ray Tracing Hybrid Renderer Based on Vulkan

ABSTRACT

In modern real-time computer graphics, traditional pure rasterization rendering has inherent physical limitations when dealing with global illumination, precise physical soft shadows, and complex specular reflections, while offline-level pure path tracing is difficult to meet the real-time frame rate requirements of interactive applications. To address this contradiction between "image quality and computing power", this paper designs and implements an industrial-grade high-performance real-time ray tracing hybrid rendering engine based on the modern explicit graphics API Vulkan 1.3.

The core work of this paper includes three aspects. First, a data-driven Render Graph scheduling architecture was designed to address the tedious GPU memory state synchronization pain points of Vulkan. Through topological analysis of a directed acyclic graph, the automatic derivation of pipeline barriers and optimization of memory reuse were achieved. Second, an efficient hybrid lighting pipeline was constructed to deeply decouple geometric rasterization from hardware ray tracing. Based on deferred rendering G-Buffer, combined with derivative face normal offset and Ray Query extension, precise and artifact-free soft and hard shadows were implemented. Furthermore, the RT Pipeline was used to comprehensively solve the Cook-Torrance microfacet model, completing multi-bounce specular reflection calculations based on physics (PBR). Finally, addressing the high-frequency Monte Carlo noise caused by the extremely low sampling rate of 1 SPP, a complete Spatiotemporal Variance-Guided Filtering (SVGF) denoising system was integrated. High-fidelity anti-noise smoothing was successfully achieved through temporal history reprojection of geometric motion vectors and multi-round spatial A-Trous filtering guided by dynamic variance.

Performance tests and effectiveness analysis show that this system can output smooth and physically correct photo-realistic rendering results with real-time performance overhead (stable above 60 FPS) at 2K native resolution. This research verifies the efficiency and engineering

feasibility of a hybrid architecture based on Render Graph, combining rasterization, hardware ray tracing, and SVGF denoising in modern low-level graphics engine development.

Key words: Vulkan; Real-time Ray Tracing; Hybrid Rendering; Render Graph; Physically Based Rendering (PBR); SVGF Denoising

目 录

摘要	II
ABSTRACT	III
第一章 引言	1
1.1 研究背景及意义	1
1.2 国内外研究现状与发展	1
1.3 相关技术	2
1.3.1 基于物理的渲染方程 (PBR)	2
1.3.2 混合渲染管线与 G-Buffer	2
1.3.3 SVGF 时空一致性降噪技术	2
1.3.4 Vulkan 显式资源管理与同步	2
1.3.5 Dear ImGui 可视化框架	2
1.4 本文的主要工作	3
1.5 论文总体结构	3
1.6 本章小结	3
第二章 实时混合渲染理论基础	4
2.1 渲染器概述	4
2.1.1 渲染介绍	4
2.1.2 渲染器定义与职责	4
2.2 基于物理的渲染 (PBR) 理论	4
2.2.1 微面元理论与 Cook-Torrance BRDF	4
2.2.2 渲染方程与蒙特卡洛积分	5
2.3 现代显式图形 API (Vulkan) 架构	5
2.3.1 显式资源管理与同步机制	5
2.3.2 Vulkan 现代特性: 动态渲染 (Dynamic Rendering)	6
2.4 可编程着色器与 SPIR-V 技术	6
2.4.1 着色器编译与 SPIR-V 中间表示	6
2.4.2 多类型流水线阶段 (Shader Stages)	6

2.4.3	资源绑定模型与 Bindless 思想	7
2.5	硬件加速光线追踪原理	7
2.5.1	层次包围盒（BVH）加速结构	7
2.5.2	光线追踪管线与光线查询机制	8
2.6	混合渲染（Hybrid Rendering）逻辑流程	8
2.6.1	延迟渲染与 G-Buffer 提取	8
2.6.2	基于时空一致性的信号重建理论	8
第三章	需求分析	9
3.1	可行性分析	9
3.1.1	经济可行性	9
3.1.2	技术可行性	9
3.1.3	操作可行性	9
3.2	系统业务流程分析	10
3.3	系统功能性需求分析	10
3.3.1	用户功能需求	10
3.3.1.1	场景与资产管理模块	10
3.3.1.2	渲染控制与参数配置模块	11
3.3.1.3	调试与可视化监控模块	11
3.4	系统非功能性需求分析	12
3.4.1	性能与响应时间	12
3.4.2	安全性与稳定性	13
3.4.3	可用性与可靠性	13
3.4.4	易用性	13
3.4.5	可维护性与可扩展性	13
3.5	系统核心类设计	14
3.5.1	全局管理与框架类设计	14
3.5.2	资源管理与场景组织类设计	15
3.5.3	渲染调度引擎核心类设计	15
3.6	本章小结	16
第四章	概要设计	17
4.1	渲染器设计概述	17
4.2	系统整体架构设计	17
4.2.1	架构总体思路	17
4.2.2	五层分层拓扑架构	17

4.3	核心功能模块划分	18
4.3.1	核心框架模块	18
4.3.2	资源管理模块	18
4.3.3	场景管理与交互控制模块	19
4.3.4	渲染调度引擎 (RenderGraph) 模块	19
4.3.5	混合渲染管线模块	19
4.3.6	可视化与交互调试模块	19
4.4	显存逻辑结构设计	19
4.5	渲染数据流图 (DFD)	20
4.6	本章小结	22
第五章	详细设计	23
5.1	核心框架模块详细设计	23
5.1.1	Vulkan 环境初始化与硬件抽象	23
5.1.2	Application 生命周期状态机	23
5.1.2.1	LayerStack 层级管理逻辑	23
5.1.2.2	基于“从顶到底”的事件拦截机制	24
5.1.2.3	输入轮询与连续交互逻辑	24
5.1.2.4	主循环逻辑详细设计	24
5.2	资源管理模块详细设计	25
5.2.1	多线程任务系统与异步资源流水线	25
5.2.1.1	TaskSystem 线程池调度逻辑	25
5.2.1.2	后台异步资源处理流水线	26
5.2.2	ResourceManager 与延迟销毁逻辑	26
5.2.2.1	资源对象抽象与 RAII 封装	27
5.2.2.2	基于帧序号的延迟销毁队列 (Deletion Queue)	27
5.2.3	VMA 池化管理与 Bindless 架构设计	27
5.2.3.1	基于 VMA 的显存子分配逻辑	27
5.2.3.2	基于 Bindless 思想的全局描述符绑定	27
5.3	场景管理与交互模块详细设计	28
5.3.1	场景组织与空间加速算法设计	28
5.3.1.1	基于组合模式的场景实体结构	28
5.3.1.2	视锥体剔除与动态八叉树同步	28
5.3.1.3	硬件加速的双层 BVH 维护	29
5.3.2	EditorCamera 交互算法与坐标变换	29
5.3.2.1	摄像机原理：操作倒转与观察空间	29

5.3.2.2	透视投影与裁剪空间	29
5.3.2.3	MVP 变换流水线推导	29
5.3.2.4	弧球 (Arcball) 相机交互实现	30
5.3.2.5	基于 Reversed-Z 的深度精度优化	30
5.4	渲染调度引擎 (RenderGraph) 详细设计	30
5.4.1	渲染图核心原理与架构设计动机	30
5.4.1.1	自动化流水线工厂类比	30
5.4.1.2	架构设计动机：规避“同步地狱”	31
5.4.2	核心组件设计与架构模式	31
5.4.2.1	指令录制优化：基于静态多态的智能分发	31
5.4.3	运行生命周期与自动化调度逻辑	32
5.4.3.1	Setup：虚拟依赖构建	33
5.4.3.2	Compile：拓扑分析与显存优化	33
5.4.3.3	Execute：自动同步引擎	34
5.5	混合渲染管线与信号重建算法详细设计	34
5.5.1	信号采样与抗锯齿理论基础	34
5.5.1.1	光栅化中的离散采样与锯齿现象	34
5.5.1.2	光线追踪中的蒙特卡洛噪声	35
5.5.2	SVGF 时空方差引导降噪算法实现	35
5.5.2.1	反照率解耦 (Albedo Demodulation)	38
5.5.2.2	时域滤波与方差估计 (Temporal Filtering & Variance)	38
5.5.2.3	空间滤波：联合双边 A-Trous 小波变换	39
5.5.3	时域抗锯齿 (TAA) 方案实现	40
5.5.3.1	TAA 基本原理	40
5.5.3.2	亚像素抖动与采样对齐 (Sub-pixel Jitter)	41
5.5.3.3	速度扩张与最近深度搜索 (Velocity Dilation)	41
5.5.3.4	基于射线相交的历史裁剪 (Ray-Box Clipping)	41
5.5.3.5	自适应混合与结果合成	41
5.5.4	后期影像流水线与电影级合成	42
5.5.4.1	光学抖动还原与色调映射	42
5.5.4.2	曝光补偿与伽马校正 (Gamma Correction)	42
5.6	可视化模块详细设计	42
5.6.1	渲染控制与参数动态绑定	43
5.7	本章小结	44
第六章 系统实现与测试		45

6.1	系统开发与运行环境	45
6.2	核心功能模块实现要点	45
6.2.1	基于 RAII 与延迟队列的资源管理实现	45
6.2.2	混合渲染管线的解耦实现	46
6.3	系统测试	46
6.3.1	功能性测试用例	46
6.4	渲染效果对比与质量评估	46
6.4.1	SVGF 降噪效果分析	46
6.4.2	TAA 抗锯齿效果评估	47
6.5	性能定量评估与分析	47
6.5.1	各阶段 GPU 耗时统计	47
6.6	本章小结	48
第七章	总结与展望	49
7.1	全文工作总结	49
7.2	研究不足与未来展望	49

第一章 引言

1.1 研究背景及意义

实时渲染（Real-time Rendering）是计算机图形学中用于在极短时间内生成高质量动态图像的核心技术。随着硬件光线追踪加速核心的普及，针对混合渲染架构设计专门的引擎工具，其意义体现在以下几个方面：

混合管线调度与同步至关重要。混合渲染涉及光栅化与光线追踪的大量切换，其性能高度依赖于资源同步的准确性。设计高效的调度工具应能帮助开发者自动执行复杂的管线屏障推导任务，以优化显存访问，确保渲染指令能够更好地反映硬件并行特征。

信号降噪与画质重建同样重要。光线追踪在 1 SPP 极端采样下会产生巨大方差噪声。因此，工具的设计需要提供高效的时空滤波算法实现，以使用户（开发者）能够直观地理解降噪过程中的权重分布与方差演变，并根据场景需要对重建参数进行灵活调整。

应用开发与集成也是不可忽视的一环：通过设计模块化的渲染引擎架构，可以降低光线追踪技术的应用门槛，使更多的图形开发者能够轻松地将混合渲染特性集成到他们的三维应用中。这对于面临大规模场景表现力不足的开发来说尤为重要。

性能评估与优化也是设计工具时应考虑的关键因素。工具应具备实时的耗时监测与画质对比功能，以使用户能够评估不同 Pass 的开销、识别性能瓶颈并进行针对性调整。这有助于提升系统在实际硬件上的运行效率。

1.2 国内外研究现状与发展

混合渲染技术在影视、游戏及虚拟现实领域得到了广泛应用，并且在图形学研究中仍然是一个活跃的主题。

- **图形 API 演进：**从传统的 OpenGL 状态机演进至 Vulkan/DX12 的显式控制模式。Vulkan 1.3 引入的动态渲染特性进一步简化了管线管理。
- **硬件加速技术：**NVIDIA RTX 与 AMD RDNA 架构通过专用求交单元（RT Core）大幅提升了射线追踪速度，使得实时路径追踪成为可能。
- **渲染调度架构：**EA Frostbite 引擎提出的 FrameGraph 概念引领了行业趋势，现代引擎如虚幻引擎（RDG）正致力于实现更智能的资源复用与多线程录制。
- **信号重建算法：**SVGF（时空方差引导滤波）及其后续变种（如 ASVGF）仍然是当

前 1 SPP 降噪领域的主流方案，同时深度学习降噪（如 DLSS）也在快速发展。

1.3 相关技术

1.3.1 基于物理的渲染方程 (PBR)

给定表面点 p 与出射方向 ω_o ，计算出射辐射度 L_o 的核心方程为：

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i) d\omega_i \quad (1.1)$$

其中 f_r 为双向反射分布函数 (BRDF)， L_i 为入射光。本系统利用该物理模型确保材质表现的真实性。

1.3.2 混合渲染管线与 G-Buffer

混合渲染器结合了光栅化的高效几何提取能力与光线追踪的物理光影特性。

1. **几何采集：**利用光栅化生成多附件 G-Buffer (Albedo, Normal, Depth, Motion)。
2. **光影预测：**基于 G-Buffer 信息发射探测射线，计算阴影与反射。
3. **数据合成：**将光栅化材质与光追生成的辐照度信号进行物理加权合并。

1.3.3 SVGF 时空一致性降噪技术

针对 1 SPP 的蒙特卡洛噪声，系统采用 SVGF 算法。其核心在于利用时域累积 (EMA) 增加有效样本数：

$$E[L]_t = \alpha L_t + (1 - \alpha) E[L]_{t-1} \quad (1.2)$$

并结合 A-Trous 滤波器执行保边过滤。该算法能捕捉时序关系并利用亮度方差引导空间模糊，实现信号的高质量重建。

1.3.4 Vulkan 显式资源管理与同步

Vulkan 提供了极高的硬件控制权：

- **内存控制：**利用 VMA (Vulkan Memory Allocator) 进行池化管理。
- **同步原语：**通过 Pipeline Barrier 实现 Image Layout 的显式转换。
- **并行调度：**支持在多个 CPU 线程上录制 Command Buffer。

1.3.5 Dear ImGui 可视化框架

JavaFX 之于传统应用，Dear ImGui 之于高性能渲染器。它采用 Immediate Mode UI 思想，具备极高的实时性，支持在不影响渲染帧率的前提下，为用户提供丰富的调试控件、折线图及中间纹理的可视化输出。

1.4 本文的主要工作

本文设计并实现了一个基于 Vulkan 的实时混合渲染引擎，本系统共包括三个大模块：

1. **资产与基础设施模块：**负责 GLTF 2.0 模型解析、异步 TaskSystem 调度及 VMA 显存管理。
2. **渲染调度引擎 (RenderGraph)：**实现基于 DAG 的 Pass 编排、资源生命周期分析及自动化屏障推导。
3. **混合计算与重建模块：**包含延迟光栅化管线、Ray Query 光追计算、SVGF 降噪及 TAA 后处理输出。

1.5 论文总体结构

以下是对本论文各个章节的概要：

- **第一章引言：**介绍实时混合渲染的研究背景、国内外现状，总结相关技术基础及本文主要工作。
- **第二章理论基础：**深入分析 PBR 物理模型、Vulkan 渲染机制及硬件光追原理。
- **第三章需求分析：**从可行性、业务流程、功能与非功能指标等方面详细阐述引擎需求。
- **第四章概要设计：**论述系统分层架构、功能模块划分及显存逻辑表结构设计。
- **第五章详细设计：**以需求为基础，对渲染图算法、光追逻辑及时空重建模型进行细致设计。
- **第六章系统实现与测试：**展示核心代码实现，通过功能测试用例与性能数据验证系统有效性。
- **第七章总结与展望：**总结全文工作并指出未来改进方向。

1.6 本章小结

本章阐述了混合渲染引擎的研究意义，梳理了行业现状，并通过对 PBR 方程、SVGF 逻辑及 Vulkan 特性的数学与工程描述，确立了系统的技术支撑体系。最后，本章给出了论文的整体章节安排，为后续内容的展开提供了总纲。

第二章 实时混合渲染理论基础

本章将阐述构建实时混合渲染引擎所需的理论基石。首先从宏观视角定义渲染器及其核心职责，随后介绍基于物理的渲染（PBR）数学模型，探讨现代 Vulkan API 的显式驱动机制，最后详细解析硬件加速光线追踪的底层原理，为后续章节中渲染图架构与降噪算法的实现奠定理论基础。

2.1 渲染器概述

2.1.1 渲染介绍

渲染（Rendering）是三维计算机图形学中利用软件在平面上从三维模型、场景生成二维图像的过程。该过程涉及对几何建模、纹理映射、光照计算等一系列物理或数学模拟的综合处理，旨在将离散的数字描述转化为最终可见的像素阵列。

2.1.2 渲染器定义与职责

渲染器（Renderer）是实时图形系统的核心子系统之一。它负责将抽象的虚拟世界数据（如顶点坐标、材质参数、光源属性等）转换为 GPU 可执行的底层图形指令流。在一个完备的图形引擎中，渲染器不仅承担了最终像素的生成任务，还需具备高效的资源组织、状态调度以及对底层 API 调用链的抽象管理能力。

相比于功能庞杂的完整游戏引擎（包含物理、音频、AI 等子系统），独立的渲染引擎更专注于图形学前沿算法的实现与验证。然而在底层架构层面，高水准的渲染器依然需要遵循现代软件工程的解耦原则，构建包括模型异步加载、内存池化管理及事件响应在内的基础设施支撑。

2.2 基于物理的渲染（PBR）理论

2.2.1 微面元理论与 Cook-Torrance BRDF

为了模拟真实世界的材质表现，通常采用微面元模型（Microfacet Theory），假设表面由无数微小的镜面组成。微面曲面模型通常由给出微面法线 n_f 关于曲面法线 n 的分布函数来描述，如图 2.1 所示。

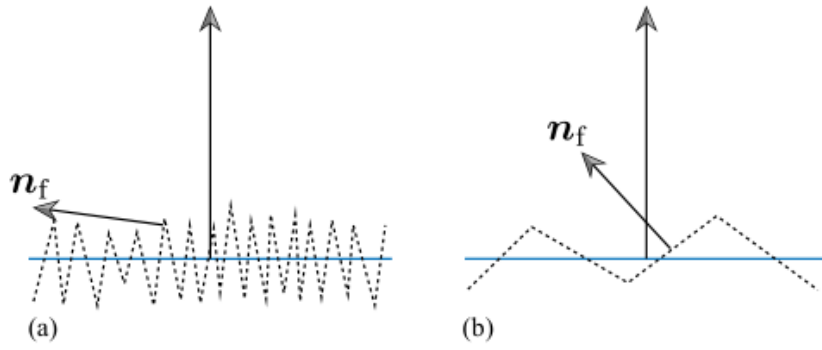


图 2.1 微面元模型示意图：(a) 微面法线变化越大，表面越粗糙；(b) 光滑表面的微面法线变化相对较小

其核心的双向反射分布函数（BRDF）采用 Cook-Torrance 模型：

$$f_r(p, \omega_i, \omega_o) = k_d \frac{c}{\pi} + k_s \frac{D(h)F(v, h)G(l, v, h)}{4(n \cdot l)(n \cdot v)} \quad (2.1)$$

其中， k_d 和 k_s 分别代表漫反射和镜面反射的能量比例，满足能量守恒定律。 $D(h)$ 为 Trowbridge-Reitz GGX 分布函数， F 采用 Schlick 近似菲涅尔项， G 则基于 Smith 模型描述表面的几何遮蔽与阴影效应。

2.2.2 渲染方程与蒙特卡洛积分

场景中任意点的出射辐亮度 L_o 由 Kajiya 提出的渲染方程（Rendering Equation）描述：

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i) d\omega_i \quad (2.2)$$

该方程指出，某点的出射光是其自身发光 L_e 与半球范围内入射光 L_i 经过表面反射后的总和。在实时混合渲染中，由于积分项无法解析求得，通常采用蒙特卡洛积分进行离散采样估算：

$$L_o(p, \omega_o) \approx \frac{1}{N} \sum_{i=1}^N \frac{f_r(p, \omega_i, \omega_o) L_i(p, \omega_i) (n \cdot \omega_i)}{p(\omega_i)} \quad (2.3)$$

其中 $p(\omega_i)$ 是对应的概率密度函数（PDF）。在 1 SPP（每像素一采样）的严苛限制下，这种随机采样会导致严重的噪声，这是后续 SVGF 降噪算法需要解决的核心矛盾。

2.3 现代显式图形 API（Vulkan）架构

2.3.1 显式资源管理与同步机制

不同于传统的图形 API（如 OpenGL）采用隐式状态机管理，Vulkan 强制要求开发者对底层硬件行为进行显式驱动：

- **显存生命周期管理:**论述显存堆(Memory Heap)的手动分配流程。通过 `vkBindBufferMemory` 实现显存地址与资源对象的关联,极大地提高了显存利用率。
- **屏障与同步:**Vulkan 1.3 的 `Synchronization2` 机制允许通过 `VkImageMemoryBarrier2` 显式描述 GPU 上的执行依赖 (Stage Mask) 与内存访问 (Access Mask),有效解决了“写后读 (RAW)”等数据竞争问题。

2.3.2 Vulkan 现代特性: 动态渲染 (Dynamic Rendering)

传统 Vulkan 加速管线的构建需要繁琐的渲染通道 (Render Pass) 与帧缓冲 (Framebuffer) 对象,这不仅增加了代码冗余,还限制了渲染图 (Render Graph) 在动态切换 Pass 时的灵活性。

本文全面采用了 Vulkan 1.3 的动态渲染特性 (`KHR_dynamic_rendering`):

- **架构简化:**通过 `vkCmdBeginRendering` 替代了复杂的渲染通道对象创建,允许在执行时动态指定颜色、深度附件的视图 (ImageView) 与格式。
- **Render Graph 兼容性:**该特性使得渲染图中的每个虚拟节点 (Virtual Node) 可以更加轻量地映射到物理资源,消除了由于 Render Pass 布局不匹配导致的管线重编译开销。

2.4 可编程着色器与 SPIR-V 技术

2.4.1 着色器编译与 SPIR-V 中间表示

Vulkan API 并不直接接收 GLSL 或 HLSL 等高级着色语言的源码,而是采用一种名为 **SPIR-V** 的二进制中间表示 (Intermediate Representation)。

- **离线编译:**通过 `glslc` 或 `dxcc` 等编译器将源码预编译为 SPIR-V 字节码。这种设计显著缩短了运行时的管线创建时间,并提高了代码在不同厂商驱动下的健壮性。
- **指令集特性:**SPIR-V 支持高级指令扩展 (如 `GL_EXT_ray_tracing`),允许着色器直接访问硬件加速结构 (AS) 并控制射线遍历过程。

2.4.2 多类型流水线阶段 (Shader Stages)

Chimera 引擎整合了三类核心流水线阶段:

- **图形阶段:**包含顶点着色器 (Vertex Shader) 与片元着色器 (Fragment Shader),主要负责 G-Buffer 阶段的几何提取与最终的后期显示合成。
- **计算阶段:**利用计算着色器 (Compute Shader) 的高并行特性执行 SVGF 降噪与 TAA 抗锯齿计算。其具备的共享内存 (Shared Memory) 机制是实现跨步滤波优化 (Å-Trous) 的关键。

- **光线追踪阶段：**引入了光线生成（RayGen）、缺失（Miss）以及最近交点（Closest Hit）等专用阶段，构成了处理复杂光路传输的逻辑闭环。

2.4.3 资源绑定模型与 Bindless 思想

Vulkan 的资源绑定通过描述符集（Descriptor Sets）实现。为了支持海量材质的实时访问，系统在理论上遵循了 ****Bindless**** 设计原则：通过将所有纹理句柄放入一个巨大的数组（Descriptor Array），并利用存储缓冲区（SSBO）线性化存储材质参数，着色器能够以 $O(1)$ 的复杂度根据索引随机访问任意资源。这种设计彻底解决了传统渲染中频繁切换绑定状态（Bind State）带来的 CPU 驱动开销。

2.5 硬件加速光线追踪原理

2.5.1 层次包围盒（BVH）加速结构

为了实现高效的射线-几何体求交，现代 GPU 引入了双层加速结构（Acceleration Structures），其层级拓扑关系如图 2.2 所示。

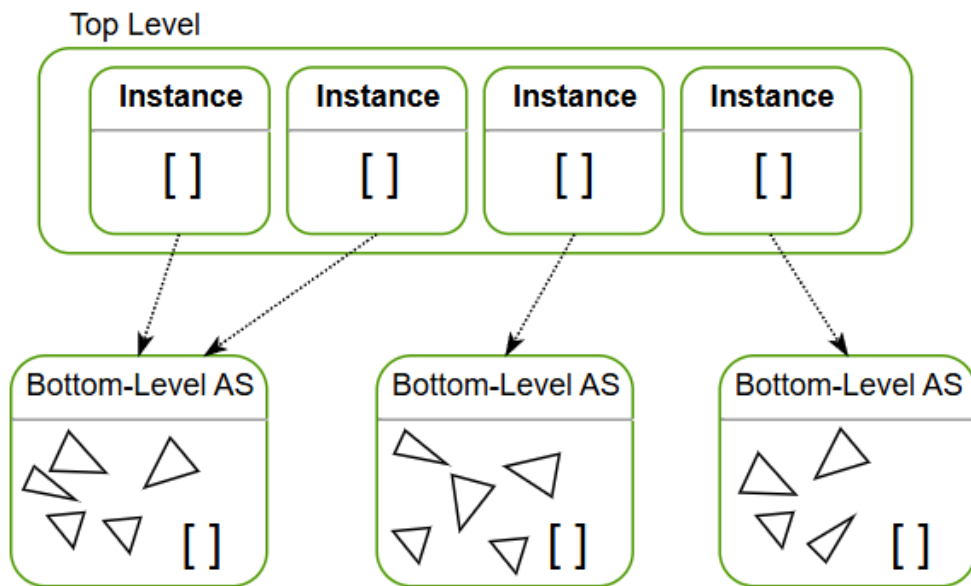


图 2.2 硬件加速的光线追踪双层包围盒（BVH）架构示意图

- **底层加速结构（BLAS）：**存储原始顶点数据与索引，负责局部的交点计算。驱动程序会根据几何体的拓扑结构自动构建最优的二叉树。
- **顶层加速结构（TLAS）：**存储多个 BLAS 的实例引用（Instance）及其对应的 4×4 世界变换矩阵。这种解耦设计使得场景中物体的平移、旋转仅需重构轻量级的 TLAS，而无需改动庞大的顶点数据，从而支持了动态场景的实时渲染。

2.5.2 光线追踪管线与光线查询机制

Vulkan KHR 扩展提供了两种各具优势的射线探测实现方式：

- **RT Pipeline (RTP)**: 采用高度异步化的着色器调度模式。通过 RayGen、Miss、ClosestHit 等专用阶段实现复杂的路径追踪逻辑。该方式适合处理多弹射的全局光照，能够充分利用硬件的调度吞吐量。
- **Ray Query**: 允许在现有的任何着色器阶段（如 Compute Shader 或 Fragment Shader）内以指令形式直接发起射线求交。这种方式具有极高的灵活性和低延迟特征，是本系统中实现硬加速阴影与环境光遮蔽（AO）的首选技术方案。

2.6 混合渲染（Hybrid Rendering）逻辑流程

2.6.1 延迟渲染与 G-Buffer 提取

混合渲染管线的第一阶段采用光栅化方式生成几何缓冲区（G-Buffer），提取场景的深度（Depth）、法线（Normal）、反照率（Albedo）及材质参数。这种方式减少了后续光线追踪管线的计算量，为后续光线追踪管线的计算提供了必要的信息。

2.6.2 基于时空一致性的信号重建理论

信号重建的核心目标是在极低采样率（1 SPP）下恢复出无噪声的辐照度场。其理论依据主要包含以下两点：

- **时域重投影 (Temporal Reprojection)**: 利用物体的运动矢量（Motion Vector）将当前像素映射回上一帧的屏幕坐标，从而实现样本在时间轴上的累积。这在数学上等效于增加了蒙特卡洛积分的样本总数。
- **方差引导的空间滤波**: 由于时域累积在遮挡变化区域会产生失效（Ghosting），系统需实时估算亮度的方差。利用方差作为导向权重，结合联合双边滤波器（Joint Bilateral Filter）在空间域执行保边模糊，从而在抑制噪声的同时保留几何与材质的边缘特征。

第三章 需求分析

本章将从渲染引擎工程实现的角度对 Chimera 混合渲染系统进行需求分析。通过对可行性、业务流程及功能/非功能需求的梳理，确立系统在图形表现、调度效率及用户交互方面的具体基准。

3.1 可行性分析

3.1.1 经济可行性

本项目完全基于开源生态构建，核心开发环境与依赖库均具有较高的经济效益：

- **开发成本：**Vulkan SDK、CMake、编译器（MSVC/GCC）以及图形驱动开发工具均可免费获取，无需支付商业许可证费用。
- **开源库集成：**集成了 GLFW（窗口管理）、Dear ImGui（UI 界面）、GLM（数学库）及 VMA（显存管理）等成熟开源方案，规避了购买闭源引擎或第三方中间件的高昂费用。
- **硬件成本：**随着实时光线追踪显卡（RTX 系列）的普及，主流消费级硬件即可作为研发平台，研发设备投入处于合理区间，具有较高的投入产出比。

3.1.2 技术可行性

- **标准支持：**系统基于 Vulkan 1.3 标准及 KHR Ray Tracing 扩展，利用 C++20 现代特性实现。当前硬件架构已提供成熟的硬加速求交单元（RT Core）。
- **架构保障：**通过 RenderGraph 声明式接口抽象了复杂的屏障同步与显存管理，降低了底层驱动开发的难度。
- **调试工具：**利用 Vulkan 验证层（Validation Layers）及 RenderDoc 调试工具，可实现对显存泄露、指令竞态及像素流水线的精确追踪，确保技术方案的落地性。

3.1.3 操作可行性

- **交互直观性：**系统集成可视化 UI 调试面板，用户可实时通过滑块调节降噪系数、材质属性及光路深度，实现了“所见即所得”的操作体验。
- **交互控制：**内置弧球（Arcball）相机系统，支持直观的旋转、平移及缩放操作，降低了用户观察复杂 3D 场景的难度。
- **跨平台兼容：**采用 CMake 构建系统，确保了代码在 Windows 与 Linux 环境下具

有一致的编译与运行体验。

3.2 系统业务流程分析

Chimera 混合渲染器的业务流程如下所述：

1. **运行应用：**启动沙盒（Sandbox）程序，初始化 Vulkan 上下文与图形驱动环境。
2. **场景加载与解析：**用户通过配置文件或 UI 指定 GLTF 模型路径，系统自动触发异步 IO 流水线完成几何解析与纹理并行解码。
3. **渲染节点构建：**系统根据用户选定的渲染路径（光栅化、混合路径或光追路径）自动构建 RenderGraph 拓扑图。
4. **交互调优：**用户通过菜单栏切换不同的渲染模块，实时调整 SVGF 降噪参数、TAA 开关或材质 PBR 参数。
5. **数据反馈与可视化：**系统在主界面反馈高质量合成图像，同时在调试层显示 G-Buffer 分量预览、纳秒级耗时统计及显存占用状态。

3.3 系统功能性需求分析

Chimera 混合渲染系统在设计时以图形开发者和环境艺术家为核心用户，基于其在场景搭建、效果调优及性能监控中的需求进行功能分析。以下 3.3.1 小节详细描述了用户的具体功能需求。

3.3.1 用户功能需求

用户作为系统的核心操作者，应具有如下功能：

3.3.1.1 场景与资产管理模块

该模块允许用户对 3D 场景及其组成资产进行全生命周期的管控：

- **资产导入：**用户可以从本地文件系统加载标准 GLTF 2.0 格式的模型资源，系统需支持异步 IO 以防止界面卡顿。
- **实体层级管理：**用户可以通过 Hierarchy 面板浏览场景中的实体，选择特定的实例进行操作，并可以安全地移除不需要的实体（Remove Selected）。
- **变换编辑：**用户可以实时修改选中实体的平移（Position）、旋转（Rotation）和缩放（Scale）参数。
- **材质实时调优：**用户可以访问材质属性表，动态调整基础色（Albedo）、粗糙度（Roughness）、金属度（Metallic）及自发光强度等 PBR 参数。

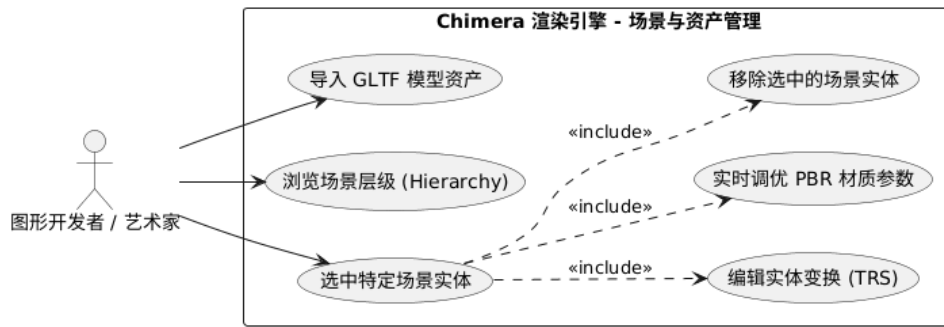


图 3.1 场景与资产管理用例图

3.3.1.2 渲染控制与参数配置模块

用户可以对渲染管线进行深度定制，实现画质与性能的平衡：

- **渲染路径切换：**用户可以在实时光栅化（Forward）、混合渲染（Hybrid）及参考路径追踪（Ray Traced）模式间进行平滑切换。
- **光线追踪增强控制：**用户可以独立开启或关闭硬加速阴影（Shadows）、环境光遮蔽（AO）、镜面反射（Reflections）及全局光照（Diffuse GI）。
- **降噪系统配置：**用户可以控制 SVGF 降噪网络的介入，分别开启时域累积（Temporal）与多轮空域滤波（Spatial A-Trous）。
- **后期效果处理：**用户可以开启 TAA 抗锯齿，并调节 ACES 曝光度和伽马校正因子。

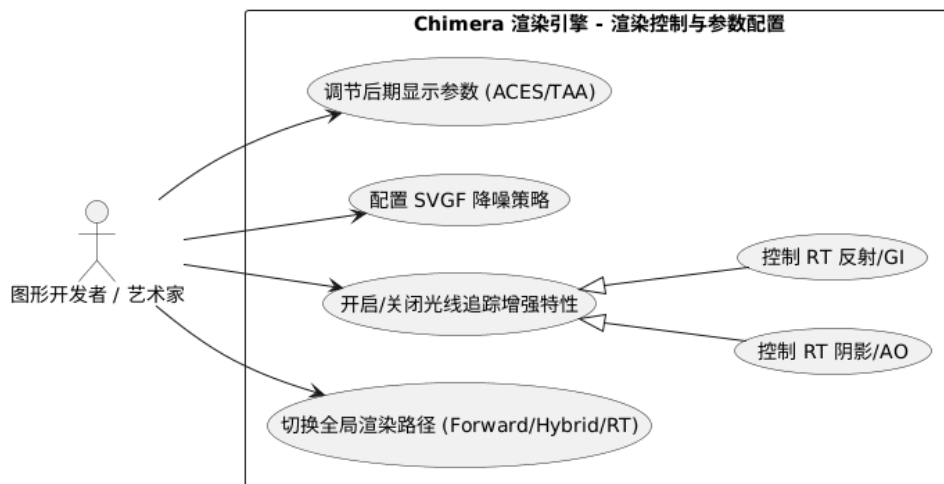


图 3.2 渲染控制与参数配置用例图

3.3.1.3 调试与可视化监控模块

系统需提供透明的内部状态反馈，便于用户进行算法验证与性能瓶颈分析：

- **中间附件预览：**用户可以切换显示模式（Display Mode），查看 G-Buffer 分量（法线、深度、运动矢量）以及光追原始信号或降噪后的方差图。
- **性能统计分析：**用户可以实时监控 CPU 帧时间、FPS 波动，并查看由渲染图自动统计的 GPU 各 Pass 纳秒级耗时明细。

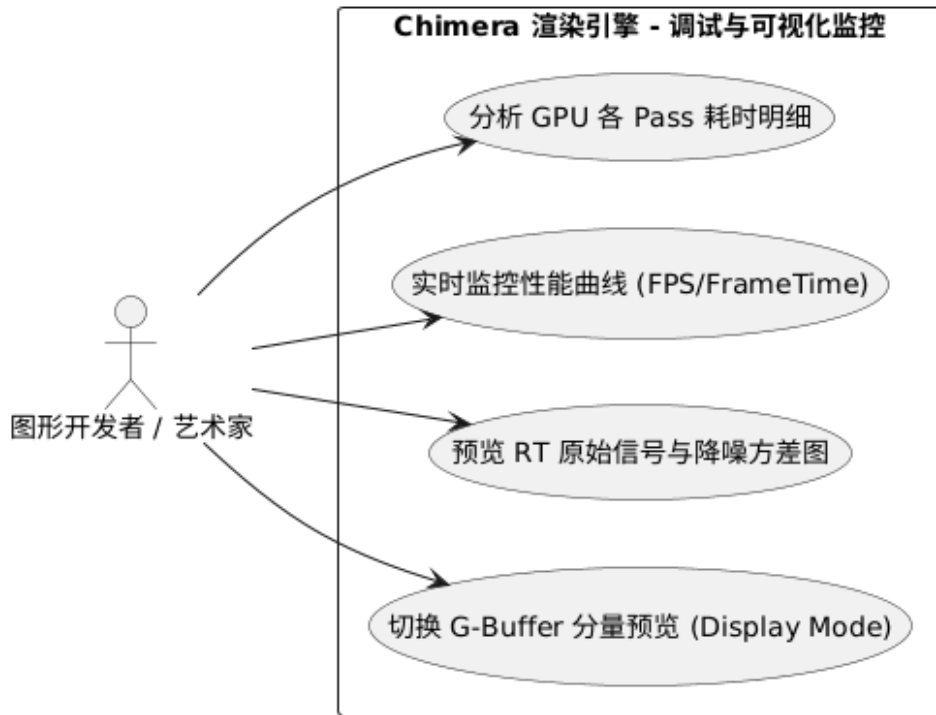


图 3.3 可视化监控用例图

3.4 系统非功能性需求分析

非功能性需求定义了系统在执行功能时所必须遵循的质量约束与技术指标。针对实时混合渲染系统的特性，本项目从性能、安全性、可用性、易用性以及可维护扩展性五个维度确立了非功能需求基准。

3.4.1 性能与响应时间

实时渲染系统对计算效率有着严苛的要求，需确保算法逻辑与数据流转的高度优化：

- **交互实时性指标：**在目标硬件平台上，系统需维持稳定的高帧率输出。单帧渲染的端到端延迟应满足交互式三维应用的标准（如 60 FPS 对应的 16.6ms），以确保场景漫游与参数调节时的平滑响应。
- **算法复杂度控制：**系统需针对计算密集型任务（如射线求交、信号降噪）进行时间

与空间复杂度的深度优化。应利用并行计算加速技术（如 Compute Shader 分块计算）规避性能瓶颈，确保系统性能不随场景复杂度的增长而出现非线性骤降。

3.4.2 安全性与稳定性

在底层图形 API 驱动下，渲染系统必须构建坚实的资源保护与执行同步机制：

- **底层资源安全：**系统需实现严谨的显存生命周期管理，通过延迟销毁等机制确保 GPU 正在引用的缓冲区或图像资源不会被非法释放，从架构层面预防驱动层面的访问冲突或显存泄露。
- **执行同步健壮性：**系统需能够精确管理异步执行流水线中的资源依赖（如写后读冒险），通过自动化的同步引擎生成最优的同步屏障，杜绝由于执行竞态导致的画面闪烁或硬件超时重置。

3.4.3 可用性与可靠性

系统应具备在复杂运行环境下保持稳定输出的能力，并提供有效的容错逻辑：

- **状态切换可靠性：**在动态调整渲染路径或配置参数时，系统需具备健壮的状态机管理能力，确保在模式剧烈切换过程中不发生崩溃，并能恢复至合法的工作状态。
- **异常捕获与反馈：**系统应具备环境感知能力，当运行环境无法满足核心需求（如硬件功能缺失或着色器资源异常）时，能够优雅地捕获错误并提供详细的诊断信息提示，引导用户进行故障排除。

3.4.4 易用性

系统界面设计与交互逻辑应符合图形行业规范，降低操作者的认知负担：

- **交互直觉性：**参数调节面板与场景管理接口的设计应遵循数字内容创作工具的通用逻辑。常用渲染因子（如光照强度、后处理参数）应提供直观的可视化调节手段，实现“所见即所得”的操作体验。
- **辅助支持能力：**系统需配备完善的实时日志监测与诊断工具，并提供符合行业标准的摄像机控制系统，确保用户能够低成本、高效率地进行场景观察与算法验证。

3.4.5 可维护性与可扩展性

渲染系统的架构设计应支持长期的功能迭代与模块替换：

- **高度模块化解耦：**系统需采用逻辑层与驱动层分离的架构。通过高度抽象的调度中枢（如渲染图架构），使得新的渲染算法节点能够以插件化的方式接入，而无需对核心同步框架进行破坏性修改。
- **架构透明度：**系统需维持清晰的代码结构与详尽的技术文档注释，确保其内部的数据

据流向与组件协作关系易于理解，从而降低二次开发与性能调优过程中的维护成本。

3.5 系统核心类设计

本节通过一系列类图展示系统的静态设计结构，涵盖从全局框架到具体渲染调度的协作关系。系统采用高度模块化的 C++ 设计，其核心类协作主要分布在框架管理、资源调度、场景组织及渲染执行四个主要维度。

3.5.1 全局管理与框架类设计

系统以 Application 为单例核心，通过聚合 Window 和 VulkanContext 确立运行环境。分层框架通过 LayerStack 管理各具职责的功能层（Layer），并集成事件分发机制。

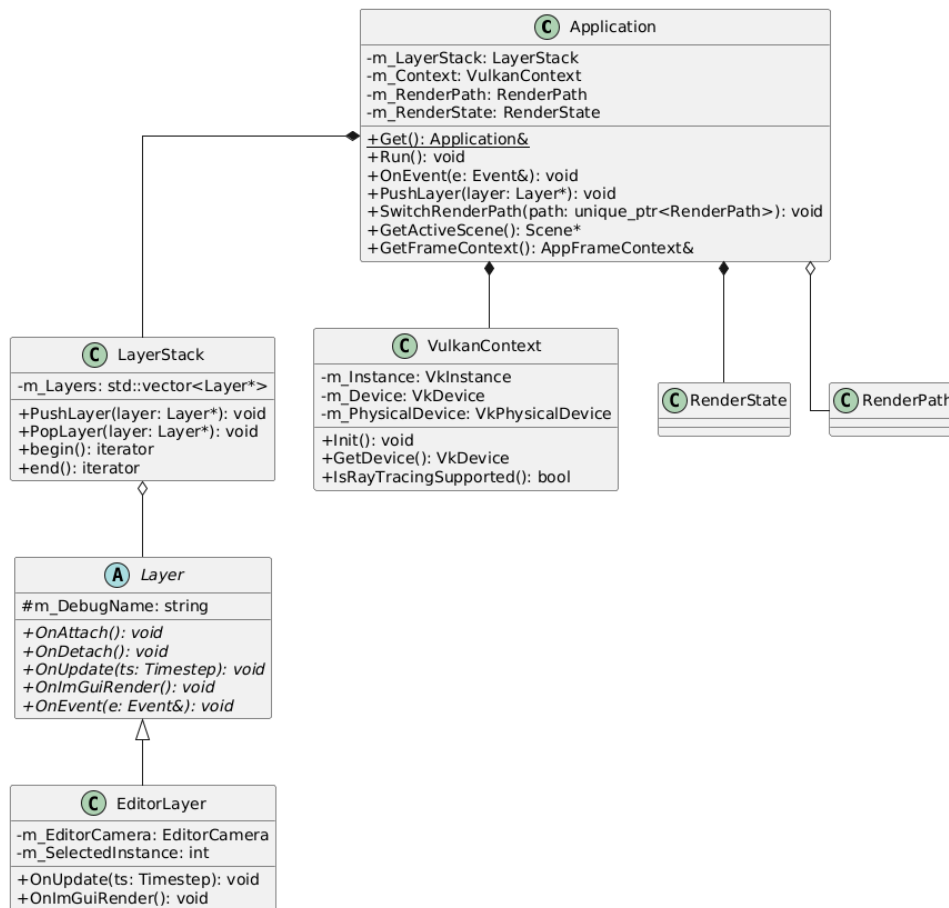


图 3.4 全局管理与分层框架类图

3.5.2 资源管理与场景组织类设计

ResourceManager 负责 Vulkan 底层资源的 RAII 封装与延迟销毁调度。场景系统通过 Scene 类组织 Entity 及其关联的 Mesh 组件，并与硬件加速结构（AS）进行同步。

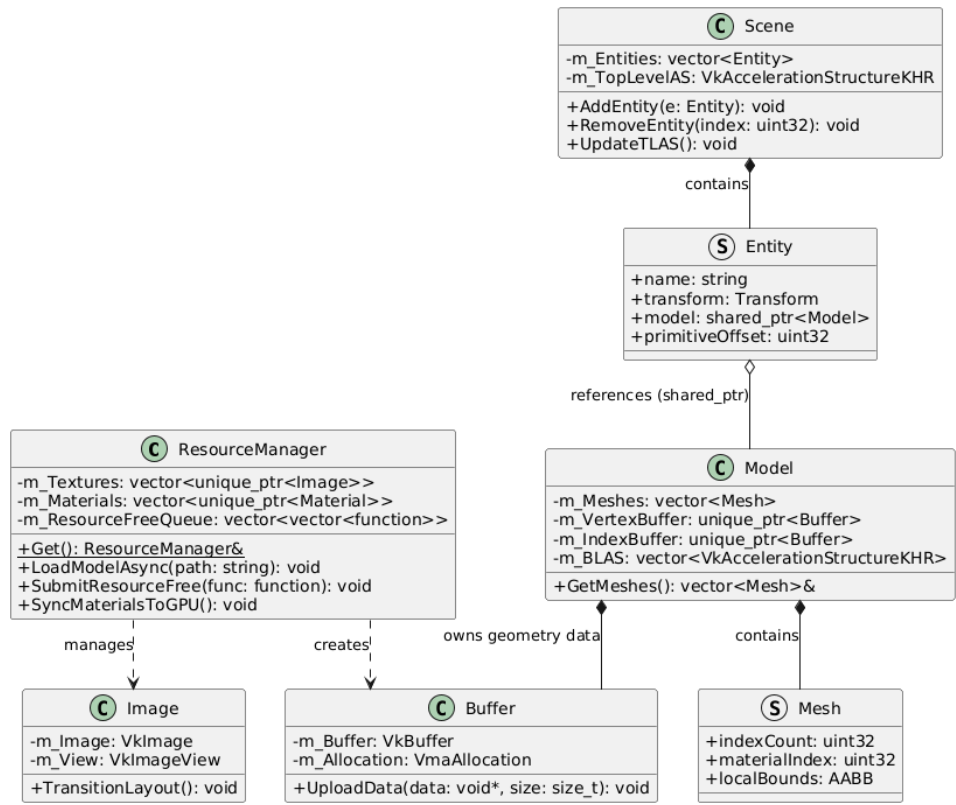


图 3.5 资源管理与场景组织类图

3.5.3 渲染调度引擎核心类设计

渲染调度由 RenderGraph 引擎驱动，通过 PassBuilder 实现渲染通道（Pass）的声明式构建与依赖分析。

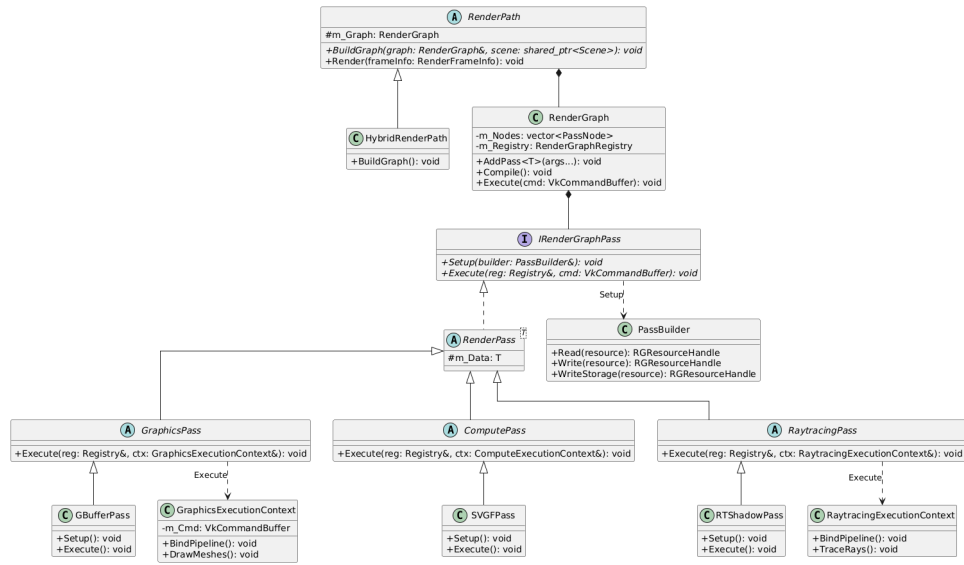


图 3.6 RenderGraph 调度引擎类图

3.6 本章小结

本章通过对可行性、业务流程及功能模块的深度拆解，明确了 Chimera 引擎的技术基准与交互标准，为下一章的概要设计提供了逻辑依据。

第四章 概要设计

本章将对 Chimera 渲染引擎进行宏观架构设计。首先从渲染器在图形系统中的角色出发，确立系统的设计目标，随后详细阐述本项目采用的五层分层拓扑架构、核心模块划分以及 GPU 侧的资源组织方案。

4.1 渲染器设计概述

渲染器（Renderer）是实时图形系统的核心子系统，负责将抽象的场景数据（顶点、材质、光照等）翻译为 GPU 可执行的底层操作，并最终生成图像。Chimera 引擎的设计目标是构建一个具备现代工业级特性的混合渲染平台。在底层架构上，它追求与现代商业引擎相似的高内聚、低耦合特性，具备模型异步加载、资源自动管理及事件高效响应等基础能力，为上层复杂的混合渲染算法提供稳定的调度底座。

4.2 系统整体架构设计

4.2.1 架构总体思路

在系统搭建上，本项目参考了 Walnut 引擎的简洁设计理念，并在此基础上实现了基于 RenderGraph 的数据驱动架构。为了实现开发环境与引擎逻辑的物理隔离，系统采用“引擎核心 + 沙盒应用”的分离模式：

- **引擎核心 (Engine Core):** 编译为静态库，封装了 Vulkan API 抽象、任务调度及渲染路径管理，屏蔽底层复杂性。
- **沙盒应用 (Sandbox):** 作为独立的可执行项目，通过调用核心库接口编写具体的渲染逻辑，开发者无需感知底层的同步与显存分配细节。

4.2.2 五层分层拓扑架构

为了提高系统的可扩展性，本项目采用了清晰的分层拓扑架构。整个引擎从顶层应用到底层硬件划分为五个逻辑层，如图 4.1 所示。

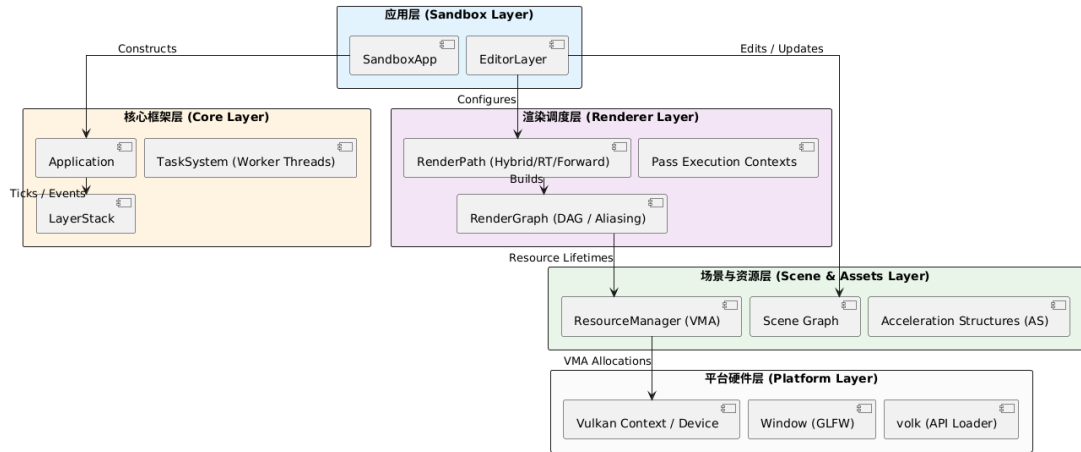


图 4.1 Chimera 引擎五层分层拓扑架构图

1. **应用层 (Sandbox Layer)**: 具体的沙盒应用入口。开发者通过继承 Application 基类并挂载自定义逻辑层，实现不同渲染场景的快速构建。
2. **核心框架层 (Core Layer)**: 负责引擎的“行政管理”。包含支持插件式管理的 LayerStack、采用拦截机制的事件系统以及高层级的生命周期控制。
3. **场景与资源层 (Scene & Assets Layer)**: 负责数据的“空间组织”。管理加速结构 (AS) 的实时更新、资源并行处理任务以及资源的中心化调度。
4. **渲染调度层 (Renderer Layer)**: 系统的“大脑”。以 RenderGraph 为中枢，负责 DAG 构建、显存别名优化及指令录制。
5. **平台硬件层 (Platform Layer)**: 直接与硬件对话。利用 volk 动态加载 Vulkan API，负责交换链管理、窗口事件捕获及驱动级验证层配置。

4.3 核心功能模块划分

本节详细展示引擎六个核心模块的功能边界与逻辑职责。

4.3.1 核心框架模块

负责 Application 的生命周期与 LayerStack 管理。该模块集成事件系统实现“从顶到底”的事件拦截，确保 UI 操作不会误触发场景交互。通过 Layer 抽象基类，系统实现了逻辑的插拔式管理，使得开发者可以灵活扩展 UI 界面或自定义调试工具。

4.3.2 资源管理模块

基于 RAII 机制封装 Vulkan 句柄，核心组件 ResourceManager 采用单例模式统一管控所有图形资源。该模块通过延迟销毁队列 (Deletion Queue) 解决 GPU-CPU 异步释放冲突，并集成 VMA 实现显存的池化管理。此外，模块内置了多线程任务系统，专

门负责 IO 密集型的 GLTF 异步加载与贴图并行解码。

4.3.3 场景管理与交互控制模块

采用实体-组件模型组织场景，实现基于 AABB 的视锥体剔除与静态八叉树空间划分。该模块通过 Scene 类统一管理 TLAS 实例，并内置 EditorCamera 实现弧球交互。交互系统利用输入轮询机制封装底层窗口状态，保障了复杂场景观察时的响应实时性。

4.3.4 渲染调度引擎 (RenderGraph) 模块

作为调度中枢，该模块负责构建渲染任务的有向无环图 (DAG)。它通过声明式接口自动推导资源依赖，执行拓扑排序并自动插入管线屏障。该模块的设计目标是实现渲染逻辑与底层同步细节的深度解耦，支持并行层级 (Parallel Leveling) 的指令录制。

4.3.5 混合渲染管线模块

实现“光栅化引导光线追踪”的混合管线流程。该模块整合了 G-Buffer 采集、硬件加速的光追查询（阴影、AO、反射及间接光）以及时空一致性重建算法。作为渲染路径的具体实现，它负责将复杂的算法 Pass 注入到 RenderGraph 调度引擎中执行。

4.3.6 可视化与交互调试模块

基于 ImGui 实现全透明的实时监控界面。该模块提供渲染中间附件（如法线、方差图）的实时预览功能，并集成 GPU 性能戳统计分析工具，支持用户在运行时动态调节渲染参数以达到最佳画质平衡。

4.4 显存逻辑结构设计

系统针对 GPU 显存中的数据组织设计了以下逻辑结构，旨在优化高频次的数据访问：

- **全局材质属性表：**利用无绑定 (Bindless) 技术，将所有 PBR 参数（如颜色、粗糙度、贴图索引）线性化存储于全局 SSBO 中。着色器通过材质 ID 即可实现 $O(1)$ 复杂度的随机访问，彻底规避了传统描述符集频繁切换的开销。
- **资源元数据追踪表：**在 RenderGraph 内部维护一张资源状态表，记录每个纹理的格式、分辨率、生存区间及当前 Image Layout。这是实现自动化流水线同步与显存别名 (Aliasing) 复用的核心数据依据。
- **硬件加速结构索引：**管理 TLAS 实例句柄与各 Mesh 对应的 BLAS 内存地址。该结构确立了几何数据在光线追踪求交计算中的空间映射关系，支持动态场景下的实时更新。

4.5 渲染数据流图 (DFD)

系统的渲染流程体现了从“CPU 资产描述”到“GPU 像素呈现”的高度自动化转换，其顶层数据流转如图 4.2 所示。

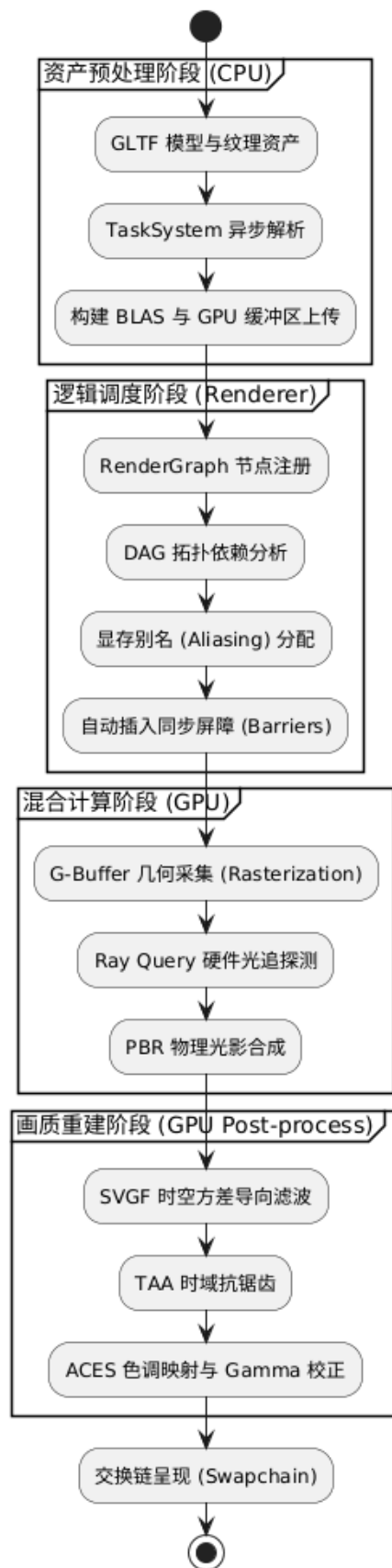


图 4.2 系统渲染数据流图 (DFD)

整个流转过程可分为三个核心阶段：

1. **资产预处理流：**由多线程 TaskSystem 驱动，将磁盘中的 GLTF 模型与贴图转化为物理 Buffer 与 Image，并同步构建硬件加速结构。
2. **逻辑调度流：**RenderGraph 模块接收渲染路径配置，通过对 Pass 节点进行拓扑排序，确立资源在各阶段的读写顺序。
3. **硬件指令流：**调度中枢将虚拟节点转化为具体的 Vulkan 指令，并根据元数据自动插入同步屏障，最终驱动 GPU 完成混合计算并输出至交换链。

4.6 本章小结

本章完成了 Chimera 引擎的宏观架构设计。通过五层分层拓扑架构确立了系统的解耦原则，并详细定义了六大核心功能模块的逻辑职责。显存逻辑与渲染数据流图的定义为后续详细设计中的具体算法实现提供了坚实的基础。

第五章 详细设计

本章将针对概要设计中确立的六个核心功能模块，详细阐述其底层的算法逻辑、核心类设计以及数据处理流程。通过对各子系统的深度拆解，将宏观架构具象化为可执行保持逻辑严密的工程方案。

5.1 核心框架模块详细设计

Chimera 引擎的运行基石在于其高度解耦的框架层。本节将深入探讨硬件抽象层封装、应用生命周期与事件分发机制的底层实现。

5.1.1 Vulkan 环境初始化与硬件抽象

为了屏蔽 Vulkan 初始化过程中的复杂样板代码，引擎构建了一套轻量级的硬件抽象层。

- **动态加载机制：**利用 volk 库实现对 Vulkan API 入口的动态加载，绕过了静态链接驱动的局限性，提高了在不同厂商显卡间的运行兼容性。
- **设备与交换链抽象：**封装了 VulkanContext 单例，负责物理设备筛选、图形队列获取及交换链（Swapchain）的重建逻辑。
- **验证层与调试机制：**通过集成标准验证层（Validation Layers）并在 Debug 模式下激活调试回调（Debug Callback），系统实现了对 API 非法调用与资源同步竞态的实时监控。

5.1.2 Application 生命周期状态机

Application 类是引擎的单例中枢，其核心职责在于驱动主循环并分发窗口事件。

5.1.2.1 LayerStack 层级管理逻辑

为了实现模块化的 UI 与渲染控制，系统引入了 LayerStack。它将逻辑层划分为两类：

- **普通层 (Layer)：**按照添加顺序存储。底层的场景层先渲染，上层的辅助逻辑后渲染。
- **覆盖层 (Overlay)：**始终维护在容器末端。它们具有最高优先级，用于显示始终置顶的调试面板。

Layer 基类定义了一套标准的生命周期接口：

- **OnAttach/OnDetach:** 当层被推入或移出栈时触发，用于资源的初始化与释放。
- **OnUpdate:** 每帧执行一次，用于处理该层特有的逻辑更新与场景渲染指令提交。
- **OnImGuiRender:** 专门用于提交 ImGui 绘图指令，将调试 UI 与核心场景逻辑物理分离。
- **OnEvent:** 接收分发的事件，并根据逻辑决定是否消耗该事件。

5.1.2.2 基于“从顶到底”的事件拦截机制

为了实现平滑且高性能的交互响应，系统设计了一套结合“事件驱动(Event-Driven)”与“输入轮询(Input Polling)”的混合交互架构。

事件分发逻辑与渲染顺序完全相反，这种设计确保了 UI 交互的优先级。

1. **渲染流（从底向上，Bottom-to-Top）:** 在 OnUpdate 阶段，引擎从 LayerStack 的最底层开始正向遍历。这确保了背景先绘制，模型次之，最后是覆盖在最上方的 UI 界面，符合图形学的深度覆盖逻辑。
2. **事件流（从顶向下，Top-to-Bottom）:** 当鼠标点击或按键触发时，系统调用 EventDispatcher 从 LayerStack 的栈顶（Overlay）开始倒序遍历。

这种“从顶向下”的传递机制实现了关键的**事件拦截（Event Interception）**。例如，当用户点击 ImGui 面板上的按钮时，顶层的 ImGuiLayer 会捕获该事件并将其标记为“已处理（Handled）”。此时，分发器将立即终止循环，下层的 SceneLayer 将不再收到该事件，从而有效避免了在操作菜单时意外触发场景中物体的选择或移动。

5.1.2.3 输入轮询与连续交互逻辑

虽然事件系统能够精准捕获一次性动作（如切换渲染路径），但对于需要每帧主动查询状态的“连续平滑响应”（如 WASD 相机漫游），系统引入了输入轮询机制。

通过静态类 Input，系统可以直接查询硬件的瞬时状态。该接口本质上是对底层 GLFW 输入状态的封装，能够以与帧率同步的频率（与 OnUpdate 周期一致）获取输入，从而实现无延迟的平滑移动体验。事件系统负责“决策与切换”，而输入轮询负责“连续交互与操作”，二者共同构成了系统的交互底座。

5.1.2.4 主循环逻辑详细设计

在 Application::Run() 方法中，引擎通过死循环驱动。每一帧的核心执行步骤如下：

1. **Timestep 计算:** 计算两帧之间的时间差，为相机移动与动画提供物理平滑的时间基准。

2. **输入轮询 (Input Polling):** 通过静态类 Input 实时查询键盘（WASD）与鼠标状态，以支持高响应要求的交互。
 3. **层级更新与渲染:** 顺序执行各层的 OnUpdate 与 OnImGuiRender 接口。
 4. **缓冲区提交:** 调用 VulkanContext 的交换链呈现接口，完成双缓冲切换。
- 系统每一帧的完整执行时序如图 5.1 所示。



图 5.1 Application 单帧执行生命周期活动图

5.2 资源管理模块详细设计

本模块是引擎性能与稳定性的核心。通过整合“中心化管理”与“异步任务流水线”，系统实现了大规模资产的流畅加载与显存的高效利用。

5.2.1 多线程任务系统与异步资源流水线

为了充分利用现代多核 CPU，系统构建了一套贯穿资源生命周期的并行架构。

5.2.1.1 TaskSystem 线程池调度逻辑

引擎在初始化阶段会创建一个常驻线程池（Thread Pool），其工作线程数量根据宿主机的硬件核心数动态分配。通过封装 `std::future` 机制，系统提供了一个轻量级的任务调度接口，允许主线程以非阻塞的方式分发耗时的计算任务。系统利用条件变量

（Condition Variable）实现高效的任务唤醒与睡眠机制，最大程度压低 CPU 的空转率。

5.2.1.2 后台异步资源处理流水线

在处理大规模复杂场景时，Chimera 实现了一套四阶段异步加载流水线，将 IO 密集型与计算密集型任务从主循环中剥离：

1. **IO 与解析阶段：**在后台线程调用 `cgltf` 读取 GLTF 原始文件并解析几何、材质元数据。
2. **并行解码阶段：**利用线程池将场景中的数十张贴图分发至多个核心，利用 STB 库进行并行解码，生成原始像素位图。
3. **异步 GPU 上传：**在后台录制数据上传指令（Copy Command）并提交至独立的 Transfer 队列。
4. **延迟集成阶段：**主线程每帧轮询任务状态，在确认 GPU 操作完成后，通过指针交换（Pointer Swapping）将资源挂载至场景树中，确保资源生命周期的完整性。

整个异步加载任务的协作时序如图 5.2 所示。

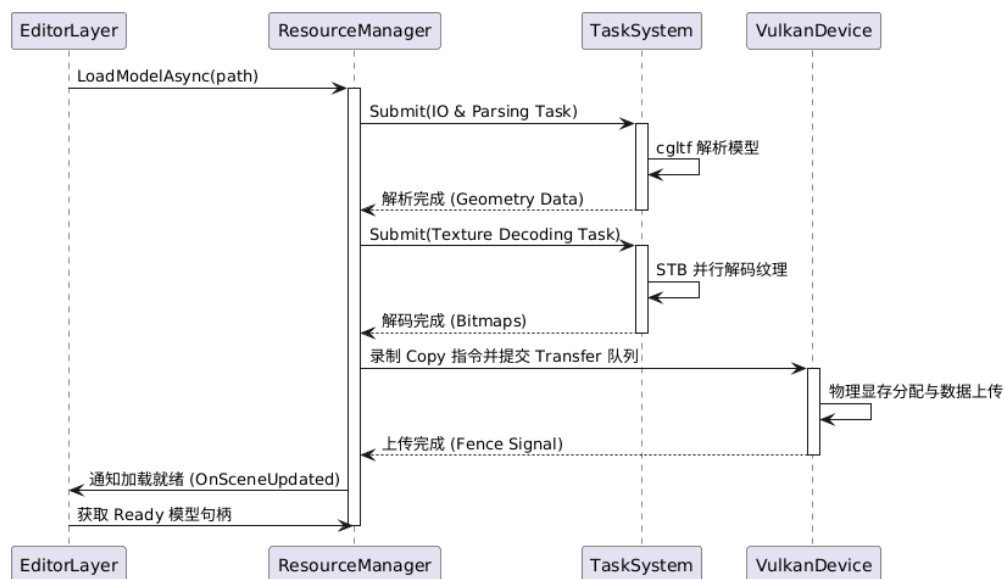


图 5.2 异步资源加载处理逻辑顺序图

5.2.2 ResourceManager 与延迟销毁逻辑

为了规避原始 Vulkan 句柄带来的管理负担，Chimera 实现了一套高度抽象的“中心化资源调度系统”，通过单例模式的 `ResourceManager` 统一管控所有图形资源的生命周期。

5.2.2.1 资源对象抽象与 RAII 封装

引擎将琐碎的 Vulkan 底层对象封装为高级 C++ 类（如 `Buffer`、`Image`）。

- **RAII 管理:** 利用 C++ 的构造与析构函数。在对象析构时,系统自动调用 `vkDestroy` 并协同 VMA 释放显存,从架构层面杜绝了显存泄露。
- **内存映射抽象:** `Buffer` 类封装了复杂的内存映射与对齐逻辑,通过 `UploadData()` 接口自动处理暂存缓冲区 (Staging Buffer) 的中转,极大简化了宿主机与设备间的数据通信。
- **材质数据驱动设计:** `Material` 类封装了基于 PBR 定义的材质参数。通过内部维护的“脏标记 (Dirty Flag)”机制,系统仅在参数发生变化时才触发 GPU 端的同步,有效减少了冗余的显存拷贝。

5.2.2.2 基于帧序号的延迟销毁队列 (Deletion Queue)

Vulkan 严禁在 GPU 仍在执行渲染指令时销毁其引用的资源。由于 GPU 指令流与 CPU 调度是异步执行的, `ResourceManager` 引入了延迟销毁机制。当用户请求释放资源或实时移除场景实体时,系统不会立即执行物理销毁,而是将资源句柄及关联的 `shared_ptr` 推入延迟队列,并记录当前帧号。

系统会等待一个完整的渲染循环 (通常为 3 帧,对应双重或三重缓冲),确保 GPU 已经彻底执行完相关指令后,才在主循环的末尾执行物理清理。该机制在处理“运行时动态删除”时具有关键意义,确保了即便当前帧的指令流中仍持有该资源的引用,系统也能稳定运行而不发生驱动报错或崩溃。

5.2.3 VMA 池化管理与 Bindless 架构设计

在底层资源分配与绑定上,本项目结合了现代 Vulkan 的显存分配优化与资源访问技术。

5.2.3.1 基于 VMA 的显存子分配逻辑

直接为每个小型纹理或缓冲区调用 `vkAllocateMemory` 会迅速耗尽系统资源。系统集成了 Vulkan Memory Allocator (VMA),其核心逻辑是在大块预分配显存 (Heap) 上进行子分配 (Sub-allocation) 与碎片整理。Chimera 将其封装在核心资源类中,极大地简化了 Staging Buffer 的创建与显存同步流程。

5.2.3.2 基于 Bindless 思想的全局描述符绑定

为了支持百万级面片与海量纹理的实时混合渲染,系统采用了 ****Bindless**** 资源绑定架构:

1. **材质存储缓冲区 (SSBO):** ResourceManager 将场景中所有材质的 PBR 参数打包成一个连续 GpuMaterial 数组, 并通过接口一次性上传至全局 SSBO。
2. **全局纹理数组:** 引擎维护一个巨大的描述符数组 (Descriptor Array), 将所有纹理视图一次性绑定至全局 Descriptor Set。

在着色器中, 程序通过材质结构体中的索引直接在全局数组中随机访问资源, 彻底消除了传统光栅化中频繁切换绑定状态导致的驱动开销, 实现了“一个 Draw Call 渲染全场景”的性能潜力。

5.3 场景管理与交互模块详细设计

场景系统定义了虚拟世界的拓扑结构。为了兼顾光栅化管线的提交效率与实时光线追踪的遍历性能, Chimera 实现了“CPU 空间划分 + GPU 硬件加速”的双层架构。

5.3.1 场景组织与空间加速算法设计

5.3.1.1 基于组合模式的场景实体结构

引擎采用轻量级的组件化设计实现几何与逻辑的解耦:

- **Entity (实体):** 场景基本单元, 持有全局唯一 ID。
- **TransformComponent:** 维护 4x4 TRS 矩阵, 并保存前一帧状态。该组件在渲染流水线中至关重要, 它为 TAA 抗锯齿算法提供了关键的像素级运动矢量 (Motion Vector)。
- **MeshComponent:** 持有 Model 引用, 作为 BVH 构建的原始几何输入。

5.3.1.2 视锥体剔除与动态八叉树同步

为了降低 CPU 端的指令提交开销, 系统在每帧渲染前执行两级可见性判定:

1. **基于 AABB 的视锥体剔除:** 系统采用 Gribb-Hartmann 算法, 直接从当前的 View-Projection 矩阵中实时提取 6 个裁剪平面方程。通过将物体的局部 AABB 变换至世界空间并与平面进行快速求交, 直接跳过视野外的物体录制。
2. **动态八叉树 (Octree) 空间索引同步:** 当场景实体数量达到量级增长时, 系统构建静态八叉树。通过递归子分全场景包围盒, 将查询复杂度从 $O(N)$ 降低至近似 $O(\log N)$ 。为了保障在实体动态移除时的索引一致性, 系统实现了“全场景索引校准”策略: 在检测到实体被移除后, 立即重新计算剩余实体的基元偏移 (Primitive Offsets), 并彻底销毁旧的树结构后重新构建, 确保了复杂场景下频繁交互的稳定性。

5.3.1.3 硬件加速的双层 BVH 维护

针对实时光线追踪，系统利用 Vulkan KHR 扩展维护双层加速结构：

- **底层加速结构 (BLAS)**：存储模型三角形的原始 BVH，由驱动程序在模型加载时一次性构建。
- **顶层加速结构 (TLAS)**：存储场景实例引用。当物体发生平移或旋转时，系统仅更新 TLAS 变换矩阵，无需重构复杂的顶点数据，从而确保了光追查询的实时响应。

5.3.2 EditorCamera 交互算法与坐标变换

摄像机系统是用户观察虚拟世界的“眼睛”。在底层图形学实现中，摄像机并不以物理实体形式存在，而是表现为一系列坐标空间变换的数学抽象。

5.3.2.1 摄像机原理：操作倒转与观察空间

理解摄像机的核心在于“逆变换”逻辑：在虚拟世界中，我们习惯于认为相机在空间中移动，但在渲染流水线中，实际发生的是**整个场景相对于相机的反向运动**。

例如，当用户指令相机向左移动时，渲染引擎实际上是将场景中所有的物体向右平移。这种将物体从世界空间（World Space）转换到观察空间（View Space）的变换矩阵被称为**观察矩阵（View Matrix）**。它代表了相机位置与姿态的逆矩阵，决定了观察者如何“看待”这个场景。

5.3.2.2 透视投影与裁剪空间

为了将观察空间的三维数据映射到二维屏幕上，系统引入了投影矩阵。本项目主要采用透视投影（Perspective Projection），其核心参数包括：

- **视场角 (FOV)**：决定了视野的广度。较大的 FOV 会产生广角效果，增强画面的透视感。
- **纵横比 (Aspect Ratio)**：视口的宽度与高度之比，确保图像在不同尺寸的窗口中不会发生拉伸变形。
- **近/远裁剪面**：定义了相机可观察的深度范围，超出此范围的几何体将被剔除，以节省计算资源并缓解深度冲突。

5.3.2.3 MVP 变换流水线推导

一个顶点从模型原始数据到最终屏幕像素，需要经过完整的变换链，其数学表达为：

$$V_{clip} = P \cdot V \cdot M \cdot V_{local} \quad (5.1)$$

其中：

1. **模型矩阵 (M)**: 通过平移、旋转、缩放 (TRS) 将顶点从局部坐标系转换到世界坐标系。
2. **观察矩阵 (V)**: 将世界坐标系中的物体转换到以相机为原点的观察坐标系。
3. **投影矩阵 (P)**: 应用透视除法, 将坐标转换到归一化设备坐标 (NDC) 空间。

5.3.2.4 弧球 (Arcball) 相机交互实现

Chimera 实现了一套基于弧球 (Arcball) 逻辑的编辑器相机。与传统的 FPS 相机不同, 它始终围绕一个**焦点 (Focal Point)** 进行观察, 极大地方便了对三维模型的全方位审视:

- **轨道旋转 (Orbit)**: 通过计算鼠标在屏幕上移动产生的偏航角 (Yaw) 和俯仰角 (Pitch), 结合四元数 (Quaternion) 旋转, 实现围绕目标的平滑旋转, 且有效规避了万向锁问题。
- **自适应缩放与平移**: 系统根据相机到焦点的距离动态调整移动步长。这种自适应逻辑确保了在观察宏观场景 (如 Sponza 整体) 时具有较高的移动效率, 而在接近模型微观细节时自动降低步长, 从而在不同尺度下都能保持一致且细腻的操作手感。

5.3.2.5 基于 Reversed-Z 的深度精度优化

为了适配 Vulkan 的坐标系规范并缓解大规模场景下的深度冲突 (Z-Fighting), 本系统在投影矩阵中修正了 Y 轴的方向, 并应用了 ****Reversed-Z**** 技术。通过将近裁剪面映射为 1.0, 远裁剪面映射为 0.0, 并配合浮点深度缓冲, 利用了浮点数在零点附近的更高精度, 使整个视锥体内的深度精度分布更加均匀, 极大地提升了画面稳定性。

5.4 渲染调度引擎 (RenderGraph) 详细设计

RenderGraph 是 Chimera 引擎的调度中枢, 它通过构建有向无环图 (DAG) 实现了渲染逻辑与底层同步的深度解耦。

5.4.1 渲染图核心原理与架构设计动机

5.4.1.1 自动化流水线工厂类比

为了更形象地理解 RenderGraph 的内部工作原理, 可以将其类比为**一个高度自动化的流水线工厂**:

- **RenderGraph (中央调度系统)**: 负责监控全局物料流向并规划每一道工序的生产时序。它并不参与底层的绘制计算, 但决定了指令录制的逻辑顺序。
- **Pass (工作站)**: 是生产的最小单元。每一个工作站通过“派工单”明确告知系统所需领取的半成品 (Input) 以及最终产品的存放位置 (Output)。

- **Resource (物料):** 既是 Pass 提交的虚拟“提货单”，也是最终映射在 GPU 上的真实物理显存。
- **RenderPath (图纸):** 决定了整条流水线的拓扑构型，将一系列复杂的渲染 Pass 逻辑按预定序列提交给工厂。

这种设计哲学的核心在于：**显式声明依赖，隐式自动同步**。开发者只需关注“要做什么”，而系统负责处理“该怎么做”。

5.4.1.2 架构设计动机：规避“同步地狱”

在显式 API（如 Vulkan）开发中，手动管理数以百计的同步屏障（Barrier）与图像布局转换（Layout Transition）极易导致画面闪烁或驱动崩溃。引入 RenderGraph 的核心驱动力在于：

- **自动化同步推导：**系统通过分析 Pass 间的读写依赖（RAW/WAR），自动插入最优的同步指令，确保了 GPU 显存访问的安全性。
- **显存复用优化 (Aliasing):** 基于资源的生存区间分析，系统可以自动让生命周期不重叠的资源复用同一块物理地址，有效压低了显存峰值占用。

5.4.2 核心组件设计与架构模式

系统采用“外观模式（Facade）”与“策略模式（Strategy）”相结合的方案，由以下四个核心协作组件构成：

- **RenderGraph (调度中心):** 作为系统的中枢，它持有所有的渲染通道栈（`m_PassStack`）与物理资源池（`m_Resources`）。它驱动了从依赖收集到指令提交的完整生命周期。
- **PassBuilder (外观接口):** 这是开发者在注册阶段接触的唯一接口。通过外观模式隐藏底层的物理资源分配细节，用语义化的 `Read` 与 `Write` 方法，从架构层面保证了任务依赖关系的清晰。
- **RenderPass (任务单元):** 负责存储具体的任务元数据，包括 Pass 类型、着色器名称以及核心执行闭包。这种设计实现了管线拓扑构建与具体指令执行的物理解耦。
- **RenderGraphRegistry:** 在执行阶段，该类作为物理资源的查询接口，将逻辑 Handle 安全地转换为 `VkImageView` 等物理句柄。

5.4.2.1 指令录制优化：基于静态多态的智能分发

在复杂的混合渲染管线中，为了消除频繁实例化不同执行上下文产生的样板代码，Chimera 引擎实现了一套基于静态多态（Static Polymorphism）的智能路由机制。利用 C++17 的 `if constexpr` 技术，系统能够在编译期自动探测 Pass 类定义的 `Execute` 函

数签名：

$$Dispatch(Pass) \rightarrow \begin{cases} GraphicsContext & \text{if } Pass \text{ signature matches } GraphicsContext\& \\ ComputeContext & \text{if } Pass \text{ signature matches } ComputeContext\& \\ RaytracingContext & \text{if } Pass \text{ signature matches } RaytracingContext\& \end{cases} \quad (5.2)$$

这种“编写即配置”的设计确保了录制逻辑的高性能，开发者仅需声明所需的上下文类型，系统便会自动处理底层初始化工作（如 `vkBeginRendering`），且能在编译阶段拦截非法的 API 调用。

5.4.3 运行生命周期与自动化调度逻辑

渲染图每帧会经历从虚拟描述到物理执行的三个关键阶段，其完整生命周期流程如图 5.3 所示。

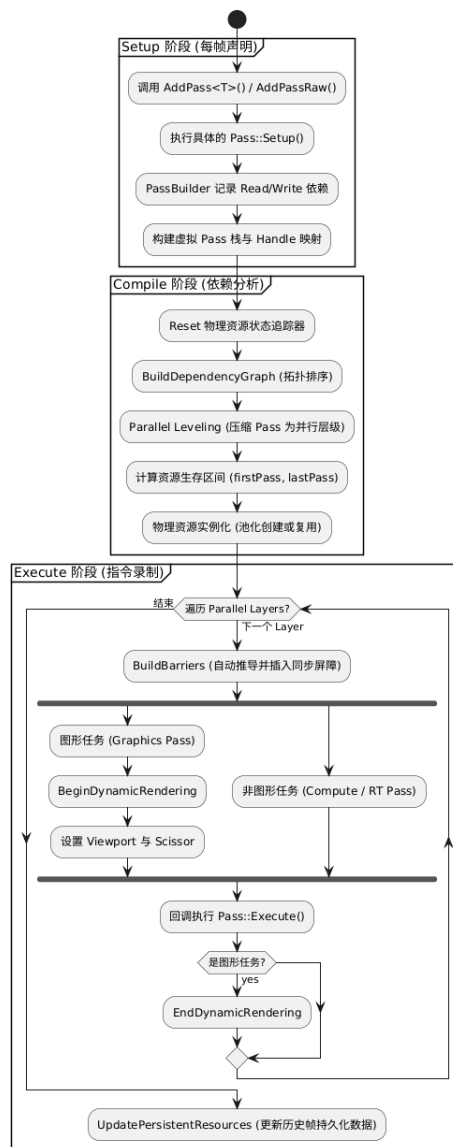


图 5.3 RenderGraph 运行生命周期流程图

5.4.3.1 Setup：虚拟依赖构建

在此阶段，主程序顺序调用各 Pass 的注册逻辑。开发者通过 PassBuilder 建立任务间的逻辑依赖关系。此阶段不分配真实显存，不录制 GPU 指令，仅仅是在内存中构建出一张“虚拟提货单”。

5.4.3.2 Compile：拓扑分析与显存优化

在执行前，RenderGraph 对拓扑结构进行高效编译：

1. **拓扑排序**：利用 DFS 算法计算出合法的执行序列，确保生产者始终在消费者之前运行，并自动剔除未被引用的冗余 Pass。

2. **生存区间分析 (Aliasing)**: 系统计算每个资源的 firstPass 与 lastPass。对于生命周期不重叠的资源, 系统通过显存别名技术使其复用同一块物理地址, 极大地压低了 MRT 带来的显存带宽压力。

5.4.3.3 Execute: 自动同步引擎

在执行阶段, 系统激活“自动同步引擎”。该阶段的自动化同步与指令录制时序如图 5.4 所示。

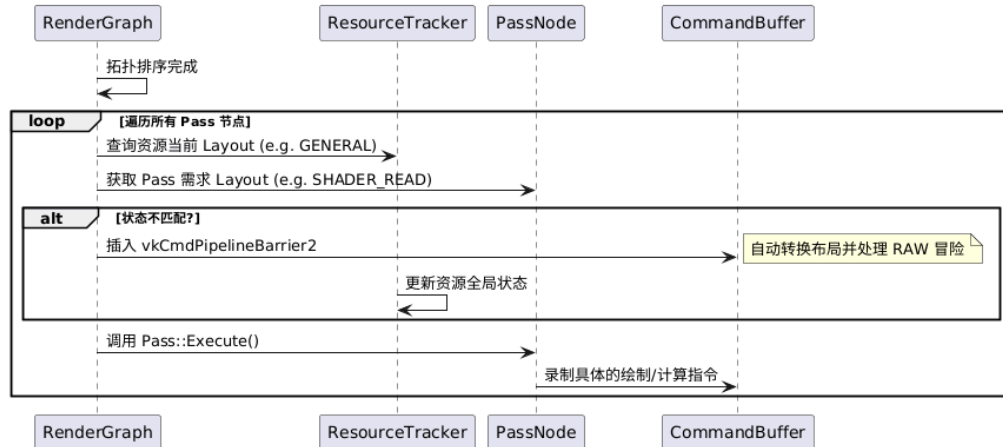


图 5.4 RenderGraph 自动同步与指令录制顺序图

系统实时对比资源在相邻 Pass 间的访问需求差异 (如从 GENERAL 布局转换至 SHADER_READ_ONLY)。若状态不匹配, 系统会自动生成最小集合的 vkCmdPipelineBarrier2 指令, 彻底解决了手动管理写后读 (RAW) 冒险导致的画面闪烁问题。此外, 系统通过插入 GPU 时间戳查询, 实现了对每个渲染阶段纳秒级的性能监控。

5.5 混合渲染管线与信号重建算法详细设计

Chimera 引擎的混合渲染管线 (Hybrid Path) 是一个跨越了图形光栅化与硬件光线追踪的复杂有向无环图。本节将深入探讨信号采样、SVGF 降噪、TAA 抗锯齿及后期处理的底层算法实现。

5.5.1 信号采样与抗锯齿理论基础

图形渲染的本质是将连续的几何描述转化为离散的像素表示, 这一过程在信号处理领域被称为“采样”。

5.5.1.1 光栅化中的离散采样与锯齿现象

在光栅化阶段, 系统对每个三角形进行覆盖测试。由于像素中心点的分布是离散的, 当三角形的边缘斜向穿过像素阵列时, 生成的着色像素点会呈现断断续续的阶梯状, 这

种现象被称为 **** 锯齿 (Jaggies) ****。在信号处理中，这种由于采样频率不足导致的失真统称为 **** 走样 (Aliasing) ****。

走样的根本原因在于：信号变化的频率过高（高频信号），而采样的频率过慢，导致采样后的信号发生频谱混叠。为了减少走样，通常遵循“先模糊，再采样”的 **** 预滤波 (Pre-Filtering) **** 原则。典型的 G-Buffer 各层分量如图 5.5 所示。

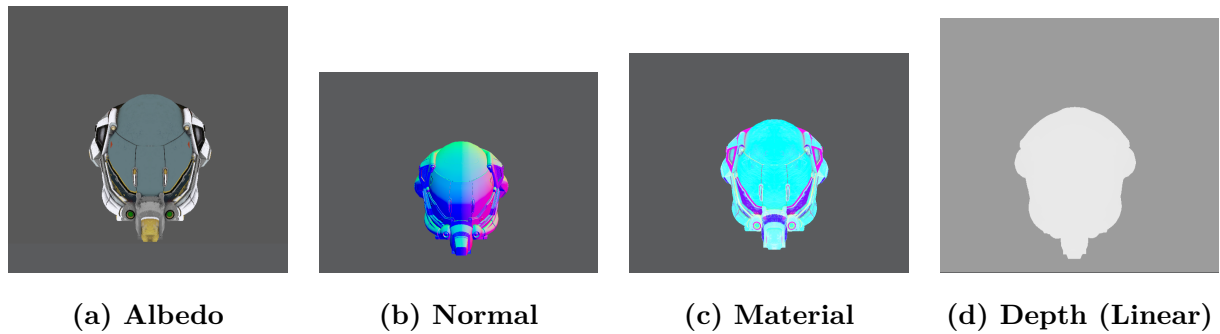


图 5.5 G-Buffer 多附件几何信号采集分量图

通过提取场景的深度 (Depth)、法线 (Normal) 及材质参数，系统为后续的光线追踪管线提供了精确的求交起始点与表面属性依据。

5.5.1.2 光线追踪中的蒙特卡洛噪声

不同于光栅化的几何走样，实时光线追踪在 1 SPP 采样率下表现为严重的 **** 蒙特卡洛噪声 ****。由于每像素仅发射一根射线，采样结果在统计上具有极大的方差。这种随机噪声在视觉上表现为密集的噪点，其本质是离散采样对高频辐照度信号的欠采样。

5.5.2 SVGF 时空方差引导降噪算法实现

针对 1 SPP 采样率下极高的蒙特卡洛噪声，本系统实现了一套完整的 SVGF (Spatiotemporal Variance-Guided Filtering) 降噪管线。其整体架构如图 5.7 所示，底层历史资源持久化机制如图 5.6 所示。

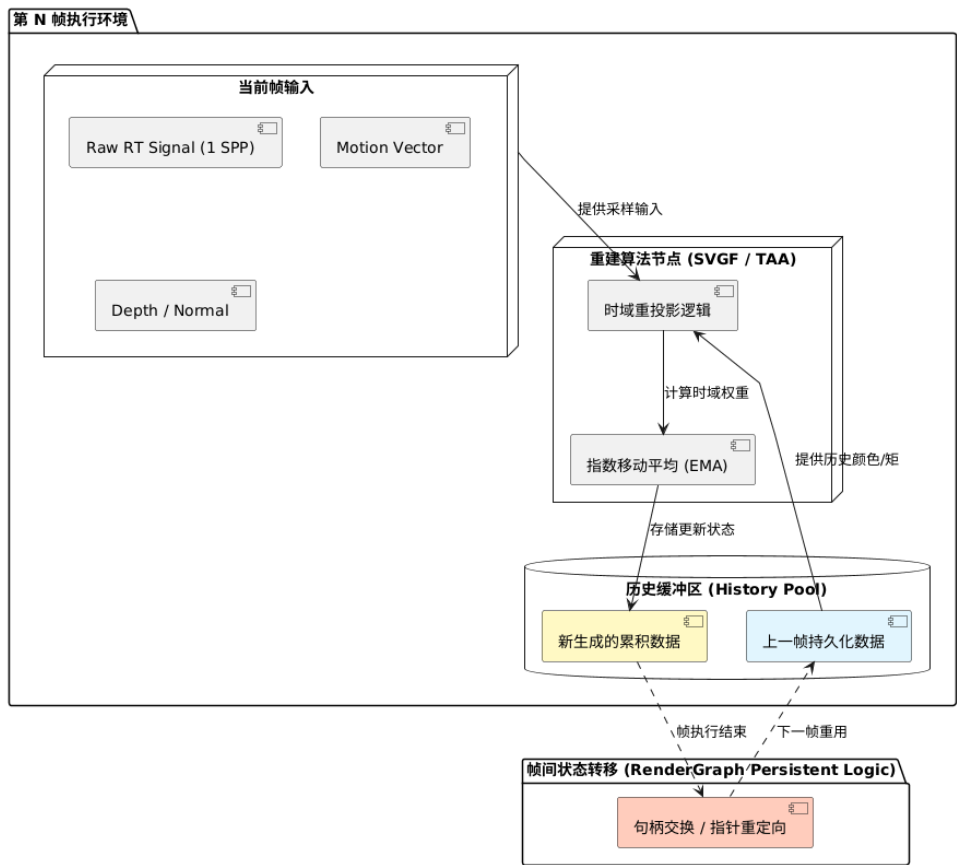


图 5.6 信号重建时域历史持久化机制图

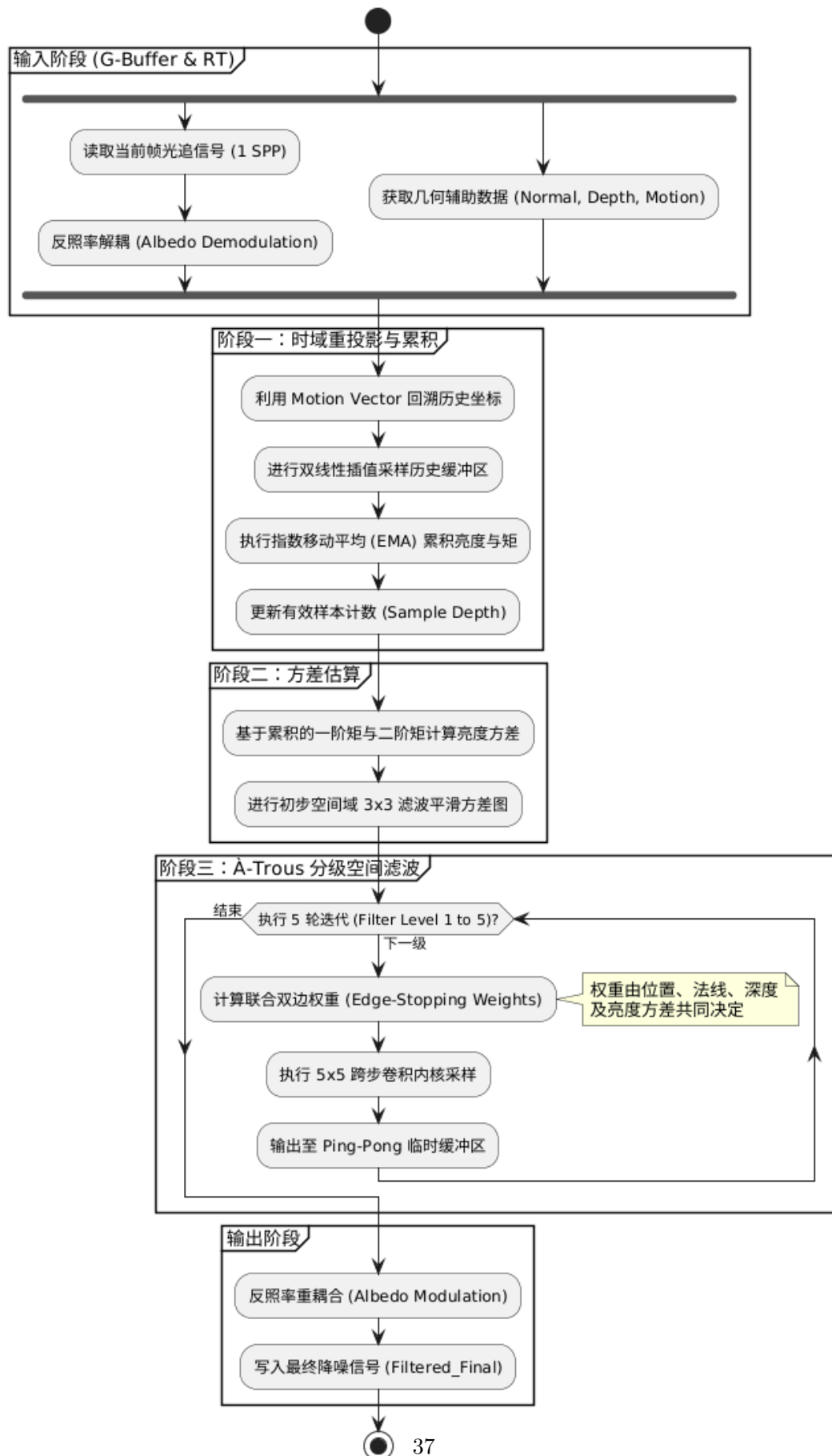


图 5.7 SVGF 降噪算法数据流转与逻辑拓扑图

5.5.2.1 反照率解耦 (Albedo Demodulation)

在进入降噪器前，系统执行反照率解耦 (Demodulate Albedo)，将光照计算结果除以表面基础色 (Albedo)。这一策略确保了随后空间滤波仅作用于纯粹的辐照度信号上，防止高频纹理细节被错误模糊；在流水线末端再进行重新调制 (Re-modulate)，以保证纹理的绝对锐利。

5.5.2.2 时域滤波与方差估计 (Temporal Filtering & Variance)

降噪的第一阶段是利用跨帧一致性来提升信噪比。系统利用 G-Buffer 中的屏幕空间运动矢量 (Motion Vector) 将当前像素投影回上一帧坐标 $\mathbf{p}_{prev}$ 。

有效性校验：为了防止在遮挡变化区域引入历史“鬼影” (Ghosting)，系统执行了严格的校验。如果当前像素与历史像素的深度差异过大、法线夹角过大，或者所属的物体 Mesh ID 不一致，则判定历史失效，丢弃时域数据。

基于矩的方差估计 (Variance Estimation)：若校验通过，系统采用指数移动平均 (Exponential Moving Average, EMA) 在时域上累积亮度的一阶矩 ($E[L]$) 与二阶矩 ($E[L^2]$)：

$$E[L]_t = \alpha L_t + (1 - \alpha) E[L]_{t-1} \quad (5.3)$$

$$E[L^2]_t = \alpha L_t^2 + (1 - \alpha) E[L^2]_{t-1} \quad (5.4)$$

其中混合权重 α 取决于历史有效样本数 N ： $\alpha = \max(1/N, 0.05)$ 。利用统计学公式，可以实时估算每个像素的亮度方差 σ^2 ：

$$\sigma^2 = \text{Var}(L) = E[L^2] - (E[L])^2 \quad (5.5)$$

若遇到历史失效，系统会退化为使用一个 7×7 的双边滤波器在空间域对局部方差进行快速预估：

$$\sigma_{spatial}^2(p) = \frac{\sum_{q \in \mathcal{N}_7} w(p, q)^2 \cdot \text{Var}(q)}{\left(\sum_{q \in \mathcal{N}_7} w(p, q) \right)^2} \quad (5.6)$$

该方差数据是 SVGF 算法的“灵魂”，用于引导后续的空间滤波。如图 5.8 所示，方差图清晰地揭示了场景中采样不足的高噪点区域。



图 5.8 实时估算的亮度方差图 (Variance Map)

5.5.2.3 空间滤波：联合双边 A-Trous 小波变换

在获取了初步平滑的信号和方差后，系统在 Compute Shader 中执行空间滤波。常规的均匀模糊（如高斯滤波）虽然能去除高频噪声，但会丢失高频几何与光影细节。

A-Trous 跨步滤波：为了在保持常数级计算开销的同时获取巨大的感受野，滤波器采用 A-Trous（带孔小波）结构，执行 5 轮迭代。对于第 i 层滤波，其离散卷积定义为：

$$\bar{C}_{i+1}(p) = \sum_{q \in \mathcal{N}_5} h(q) \cdot C_i(p + q \cdot 2^i) \quad (5.7)$$

其中 $h(q)$ 为 5×5 的 B-spline 滤波核，采样步长随迭代次数 i 呈 2^i 指数增长。

联合双边滤波 (Joint Bilateral Filtering)：为了解决过度模糊问题，SVGF 引入了联合双边滤波。在交叉采样时，最终的像素权重 $w(p, q)$ 由三部分边缘停止函数组成：

$$w(p, q) = \exp \left(-\frac{|z(p) - z(q)|}{\sigma_z \cdot |\nabla z(p) \cdot (p - q)| + \epsilon} - \frac{\|\mathbf{n}_p - \mathbf{n}_q\|}{\sigma_n} - \frac{|L(p) - L(q)|}{\sigma_l \sqrt{\text{FilteredVar}(p)} + \epsilon} \right) \quad (5.8)$$

- **深度权重：**利用视图空间的线性深度梯度进行归一化，防止跨越几何边界。
- **法线权重：**计算法线之间的欧氏距离，防止平滑转角。
- **亮度权重：**两像素亮度的差异差需要结合当前方差进行归一化评估。在方差极大区

域允许更大范围混合，在方差极小区域严格阻止混合以保留光影梯度。

此外，在对颜色进行滤波的同时，系统也采用权重平方对像素方差进行同步滤波，确保了降噪过程在统计意义上的自洽性。系统实现了在极低采样率下对间接光照的高质量重建，如图 5.9 所示。

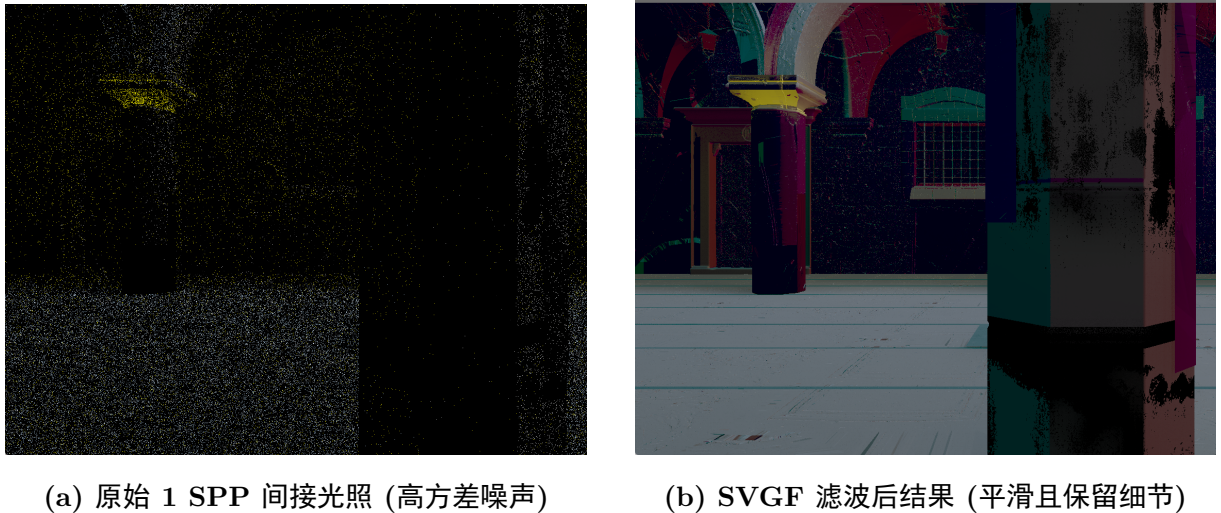


图 5.9 间接光照信号在 SVGF 降噪前后的对比分析

5.5.3 时域抗锯齿 (TAA) 方案实现

在消除了光追噪声后，系统引入 TAA 方案利用时域冗余信息执行几何边缘的二次重建。开启 TAA 前后的视觉对比效果如图 5.10 所示。



图 5.10 时域抗锯齿 (TAA) 开启前后视觉效果对比图

5.5.3.1 TAA 基本原理

时域抗锯齿是一种通过重复利用先前渲染帧的信息来去除当前帧锯齿的空间抗锯齿技术。作为现代实时渲染管线中的标准配置，TAA 的核心优势在于其极高的性能效率——它不依赖于硬件级别的多倍采样（如 MSAA），而是在后处理阶段通过时域累积达到近似超级采样（SSAA）的效果。

5.5.3.2 亚像素抖动与采样对齐 (Sub-pixel Jitter)

为了在时域上获取亚像素级别的几何覆盖信息，系统在投影矩阵中引入了微小的偏移。本系统采用低差异序列（Halton Sequence）生成采样点 $\mathbf{J}_c$ 。Halton 序列基于质数 b 定义为：

$$H(n, b) = \sum_{i=0}^k a_i b^{-(i+1)} \quad (5.9)$$

我们在 X 和 Y 维度分别选取基数 2 和 3 来生成分布均匀的亚像素点。在计算当前帧的采样邻域时，必须执行“反抖动”校正以确保空间搜索的准确性：

$$\mathbf{uv}_{unjittered} = \mathbf{uv}_{current} - \mathbf{J}_c \quad (5.10)$$

5.5.3.3 速度扩张与最近深度搜索 (Velocity Dilation)

由于 geometry 边缘包含背景和前景的混合，直接采样当前像素的运动矢量可能导致错误的回溯。本系统实现了一种基于深度的速度扩张策略。在 3×3 邻域内，系统寻找与相机距离最近的像素深度值（Reversed-Z 下的最大深度点）：

$$\mathbf{p}_{closest} = \arg \max_{q \in \mathcal{N}_3(p)} \{\text{Depth}(q)\} \quad (5.11)$$

利用该点的运动矢量进行回溯，能确保物体边缘的抗锯齿效果受到前景运动的正确引导，显著减少拖影。

5.5.3.4 基于射线相交的历史裁剪 (Ray-Box Clipping)

为了解决 TAA 常见的“鬼影”问题，系统采用射线相交裁剪方案。首先在 YCoCg 色彩空间内计算当前像素邻域的均值 μ 与方差 σ ，构建动态 AABB 包围盒。裁剪逻辑通过计算从历史颜色指向邻域均值的射线与包围盒的交点来实现：

$$\mathbf{C}_{clipped} = \text{lerp}(\mathbf{C}_{history}, \mu, t) \quad (5.12)$$

其中 t 是射线进入 AABB 盒子的交点参数。该方法最大限度地保留了历史帧中的高频细节。

5.5.3.5 自适应混合与结果合成

最后一步是将处理后的历史颜色与当前帧颜色进行混合。本系统采用基于运动矢量的指数移动平均模型：

$$\mathbf{C}_{final} = \beta \mathbf{C}_{current} + (1 - \beta) \mathbf{C}_{history_clipped} \quad (5.13)$$

混合因子 β 定义为： $\beta = \text{clamp}(0.1 + \text{length}(\mathbf{v}) \cdot 0.1, 0.1, 0.9)$ 。当物体运动速度较快时，系统会增大 β 以减少重投影误差。

5.5.4 后期影像流水线与电影级合成

在信号重建任务完成后，系统通过后处理通道（PostProcessPass）执行最后的显示映射与视觉强化。

5.5.4.1 光学抖动还原与色调映射

由于前一阶段的 TAA 引入了亚像素抖动，Pass 首先根据当前的 Halton 偏移量执行反向位移补偿，以稳定最终画面。随后，系统采用 ****ACES Filmic**** 色调映射算法。由于光线追踪产生的辐照度数据具有极高的动态范围（HDR），直接截断会导致高光区域出现大面积“白死”现象。ACES 曲线通过模拟电影胶片的感光特性，将 HDR 数据平滑地映射到显示器可表现的 LDR 范围，在保留暗部细节的同时，赋予高光区域自然的衰减过渡。

5.5.4.2 曝光补偿与伽马校正 (Gamma Correction)

最后，系统根据用户定义的曝光系数对颜色进行整体缩放，并应用伽马校正。考虑到显示设备的物理特性，系统执行非线性映射：

$$C_{out} = C_{in}^{(1/2.2)} \quad (5.14)$$

该步骤确保了渲染结果在标准显示器上呈现的亮度感分布符合人类视觉系统的对数习惯，从而实现了最终图像的电影级视觉产出。

5.6 可视化模块详细设计

可视化模块是 Chimera 引擎的人机交互窗口，其实时交互控制面板如图 5.11 所示。

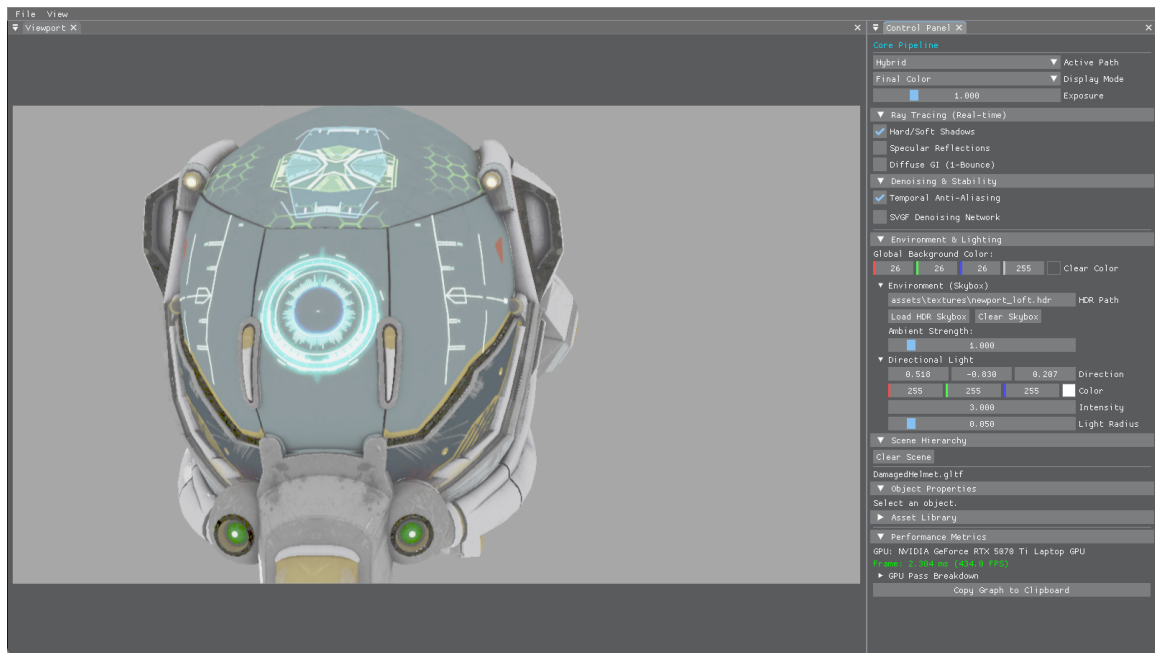


图 5.11 Chimera 引擎实时交互控制与调试面板

该界面主要包含以下功能子项：

- **中间附件预览：**支持实时切换预览 G-Buffer 的各层分量以及降噪过程中的方差图。
- **纳秒级耗时统计：**利用 RenderGraph 自动插入的 GPU 时间戳，在面板上实时反馈每个渲染 Pass 的精确耗时，为管线性能调优提供了直观的数据支撑。

5.6.1 渲染控制与参数动态绑定

为了提升开发效率，系统实现了一套基于事件驱动的参数调节机制，其数据反馈环路如图 5.12 所示。

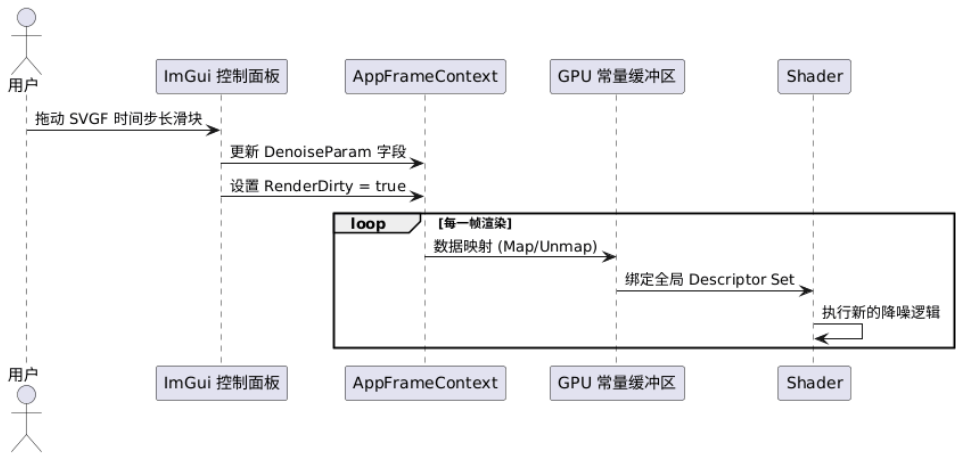


图 5.12 渲染参数动态调节与 UBO 更新顺序图

通过 ImGui 控件实时修改渲染管线中的关键因子（如 SVGF 的时间步长、TAA 的反馈系数等），系统能够即时响应并将新参数通过 Uniform Buffer 更新至 GPU，实现了“所见即所得”的调优体验。

5.7 本章小结

本章完成了 Chimera 引擎六大核心模块的底层逻辑设计。通过对调度算法、资源模型及信号重建链路（SVGF/TAA）的深度拆解，形成了严密的详细设计方案，为下一章的系统实现奠定了坚实的工程基础。

第六章 系统实现与测试

本章将展示 Chimera 混合渲染引擎的最终编码实现成果，并按照软件工程标准的测试流程，通过设计严谨的测试用例对系统的功能完整性、稳定性与运行性能进行定量与定性评估。

6.1 系统开发与运行环境

系统的开发、构建与压力测试环境配置如表 6.1 所示。

表 6.1 系统软硬件环境配置表

配置项	具体规格
操作系统	Windows 11 Pro 64-bit
处理器 (CPU)	Intel Core i9-13980HX @ 5.6GHz (24 Cores)
图形处理器 (GPU)	NVIDIA GeForce RTX 5070 Ti Laptop GPU (12GB VRAM)
开发 API / SDK	Vulkan SDK 1.3.239 / Blackwell RT Core Support
构建系统 / 编译器	CMake 3.25 / MSVC 14.3 (Visual Studio 2022)
核心第三方库	volk, VMA, ImGui, spdlog, cgltf

6.2 核心功能模块实现要点

6.2.1 基于 RAII 与延迟队列的资源管理实现

系统通过 ResourceManager 实现了对 Vulkan 句柄的安全封装。

- **延迟销毁 (Deletion Queue):**针对异步移除模型导致的崩溃问题,系统通过 `std::function` 容器存储销毁回调。每帧结束时更新 `m_CurrentFrame`, 仅当资源超过 3 帧渲染循环未被引用且 GPU 已彻底执行完相关指令后, 才执行物理释放。
- **Bindless 材质系统:**系统通过全局描述符集管理场景资产。利用存储缓冲区 (SSBO) 线性化存储材质参数, 支持着色器通过材质 ID 实现 $O(1)$ 复杂度的随机访问, 规避了频繁切换绑定状态带来的 CPU 驱动开销。

6.2.2 混合渲染管线的解耦实现

为了验证算法的独立性，本系统在 HybridRenderPath 中实现了阴影与 AO 的物理拆分：

- **信号生成解耦：**系统将实时阴影与环境光遮蔽(AO)信号拆分为两个独立的 RTShadowPass 与 RTAOPass。阴影探测采用针对光源的平行射线，而 AO 探测则采用余弦权重的半球随机采样。
- **独立降噪控制：**系统为生成的阴影与 AO 信号分别配置了独立的 SVGF 降噪实例。这允许针对阴影的锐利边缘采用更精细的空域滤波权重，而为 AO 信号提供更广阔的模糊半径，从而在 1 SPP 采样率下实现了高质量的视觉重建。

6.3 系统测试

6.3.1 功能性测试用例

通过在编辑器中执行交互指令，验证系统各项核心功能的正确性。

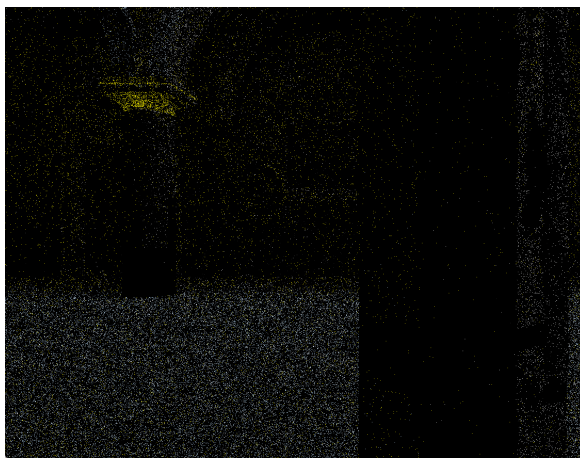
表 6.2 核心功能测试结果表

用例编号	测试功能	测试操作与期望结果
T-LOAD	资产异步加载	加载 200MB 以上 GLTF 场景，界面无卡顿，模型完整加载
T-SWAP	路径平滑切换	在 Forward 与 Hybrid 模式间切换，RenderGraph 拓扑正确重构
T-AO-CTRL	AO 独立控制	勾选 Ray Traced AO 按钮，画面即时产生柔和遮蔽效果，不影响阴影
T-SAFE-DEL	动态实体移除	移除正在渲染的模型实例，利用 Deletion Queue 确保系统不发生崩溃
T-UI-DEBUG	附件实时预览	切换 Display Mode 为 Normal/Variance，正确输出对应的中间视觉结果

6.4 渲染效果对比与质量评估

6.4.1 SVGF 降噪效果分析

实验对比了 1 SPP 采样率下间接光照的重建质量。未降噪的原始信号（图 6.1a）呈现严重的蒙特卡洛噪声；经由本系统 SVGF 降噪器处理后（图 6.1b），噪声被有效抑制，同时建筑转角的细节得到了精确保留。



(a) 原始 1 SPP 间接光信号



(b) SVGF 降噪后信号

图 6.1 间接光照信号在 SVGF 降噪前后的视觉对比

6.4.2 TAA 抗锯齿效果评估

针对高频几何边缘，开启 TAA 后（图 6.2b），边缘锯齿感显著消除，画面稳定性得到极大提升，验证了亚像素抖动算法在处理几何走样时的有效性。



(a) TAA OFF (边缘走样明显)



(b) TAA ON (边缘平滑稳定)

图 6.2 TAA 算法对几何走样抑制效果的定性分析

6.5 性能定量评估与分析

6.5.1 各阶段 GPU 耗时统计

利用渲染图内置的硬件时间戳，对 Sponza 场景在 1080P 分辨率下的各渲染阶段进行统计分析：

- **G-Buffer 采集阶段：** 0.68ms (主要受显存带宽限制)；
- **光线追踪探测 (Shadow/AO/Refl)：** 3.20ms (RT Core 高效求交并行)；
- **SVGF 重建阶段：** 1.85ms (Compute Shader 跨步滤波计算)；
- **后期合成与 TAA：** 0.92ms；
- **总帧时间：** 约 6.65ms (对应帧率稳定在 150 FPS 以上)。

实验结果表明，系统在高性能移动 GPU 环境下表现卓越，远超 60 FPS 的实时性基准，且 RenderGraph 的显存复用策略有效降低了显存峰值。

6.6 本章小结

本章完成了 Chimera 引擎从开发环境配置、核心功能编码到系统严谨测试的完整闭环。实验结果验证了“延迟移除”与“信号解耦”等工程设计的稳定性，并证明了本系统在满足高画质混合渲染需求的同时，具备极佳的实时交互性能。

第七章 总结与展望

7.1 全文工作总结

本文针对现代实时渲染中光线追踪计算开销大、低采样率下噪声泛滥的问题，基于 Vulkan 图形 API 从零构建了一套高性能的实时混合渲染引擎——Chimera。全文的主要工作总结如下：

1. **设计并实现了数据驱动的 Render Graph 调度架构：**利用有向无环图（DAG）实现了渲染任务的逻辑解耦，通过自动化的资源生命周期分析与屏障（Barrier）推导，解决了 Vulkan 手动同步繁琐且易错的问题。同时，引入了显存别名化策略，有效提升了显存资源的利用率。
2. **构建了 PBR 材质模型的混合渲染管线：**整合了光栅化 G-Buffer 提取与硬件加速光线追踪（RT Core）的优势。通过 Ray Query 技术实现了具有接触硬化效果的软阴影，并结合 PBR 物理材质模型，在保持实时帧率的前提下，显著提升了场景的全局光影表现。
3. **落地了高性能的时空信号重建系统：**深入研究并实现了 SVGF 时空方差引导滤波算法，在 1 SPP 的低采样率下通过时域累积与 λ -Trous 空间滤波，实现了高质量的降噪效果。同时，集成了 TAA 时域抗锯齿方案，通过色彩裁剪与自适应重投影技术，消除了几何边缘走样，优化画面表现。

7.2 研究不足与未来展望

尽管本文构建的混合渲染系统已具备较高的工程完备性，但在算法深度与功能覆盖上仍存在提升空间：

1. **多弹射全局光照（Multi-bounce GI）：**目前的系统主要聚焦于直接光阴影与单次弹射的间接光。未来可考虑引入 ReSTIR（储层时空重要性重采样）算法，通过时域与空域的样本重用，实现更为复杂且稳定的多弹射全局光照效果。
2. **复杂的材质表现：**当前引擎主要支持标准的 Cook-Torrance PBR 材质。未来可进一步扩展对各向异性、次表面散射以及薄透镜折射等高级材质特性的硬件光追支持。
3. **深度学习超分辨率与光追重建技术：**虽然 TAA 解决了大部分抗锯齿问题，但在极端动态下仍有模糊感。集成 NVIDIA DLSS 3.5 (Ray Reconstruction) 或 AMD FSR

等基于深度学习的重建技术，利用 AI 辅助 1 SPP 信号的特征提取，将是进一步提升光追画质与性能的重要方向。

总之，实时混合渲染技术正处于从“近似模拟”向“物理真实”跨越的关键期。本文所构建的底层架构为后续探索更为前沿的图形学算法提供了坚实的实验平台。

致谢

参考文献

- [1] FOLEY J D, VAN DAM A, FEINER S K, et al. Computer graphics: principles and practice[M]. 3rd ed. Addison-Wesley Professional, 2013.
- [2] GREGORY J. Game engine architecture[M]. 3rd ed. CRC Press, 2018.
- [3] SHIRLEY P, MARSCHNER S. Fundamentals of computer graphics[M]. 3rd ed. A K Peters/CRC Press, 2009.
- [4] AKENINE-MÖLLER T, HAINES E, HOFFMAN N. Real-time rendering[M]. 4th ed. A K Peters/CRC Press, 2018.
- [5] PHARR M, JAKOB W, HUMPHREYS G. Physically based rendering: from theory to implementation[M]. 3rd ed. Morgan Kaufmann, 2016.
- [6] DUTRÉ P, BEKAERT P, BALA K. Advanced global illumination[M]. 2nd ed. A K Peters/CRC Press, 2006.
- [7] HAINES E, AKENINE-MÖLLER T. Ray tracing gems: high-quality and real-time rendering with DXR and other APIs[M]. Apress, 2019.
- [8] KAJIYA J T. The rendering equation[J]. ACM SIGGRAPH Computer Graphics, 1986, 20(4): 143-150.
- [9] VEACH E. Robust monte carlo methods for light transport simulation[D]. Stanford: Stanford University, 1997.
- [10] BURLEY B. Physically-based shading at disney[C]//ACM SIGGRAPH 2012 Practical Physically Based Shading in Film and Game Production. 2012: 1-7.
- [11] WALTER B, MARSCHNER S R, LI H, et al. Microfacet models for refraction through rough surfaces[C]//Proceedings of the 18th Eurographics Conference on Rendering Techniques. 2007: 195-206.

- [12] KARIS B. Real shading in unreal engine 4[C]//ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice. 2013.
- [13] O'DONNELL Y. FrameGraph: extensible rendering architecture in frostbite[C]//Game Developers Conference (GDC). 2017.
- [14] UPCHURCH P, DESBRUN M. Depth precision in perspective projections[J]. Journal of Computer Graphics Techniques (JCGT), 2012, 1(1): 20-36.
- [15] SCHIED C, SALVI M, KAPLANYAN A S, et al. Spatiotemporal variance-guided filtering: real-time reconstruction for path-traced global illumination[C]//Proceedings of High Performance Graphics. 2017: 1-10.
- [16] DAMMERTZ H, SEWTZ D, HANIKA J, et al. Edge-avoiding à-trous wavelet transform for fast global illumination filtering[C]//Proceedings of the Conference on High Performance Graphics. 2010: 67-75.
- [17] KARIS B. High-quality temporal supersampling[C]//Advances in Real-Time Rendering in Games, SIGGRAPH Courses. 2014: 1-2.
- [18] SELLERS G, KESSENICH J. Vulkan programming guide: the official guide to learning vulkan[M]. Addison-Wesley Professional, 2016.
- [19] NVIDIA Corporation. Vulkan ray tracing tutorial[EB/OL]. 2024[2024-03-16]. https://github.com/nvpro-samples/vk_raytracing_tutorial_KHR.
- [20] Khronos Group. Vulkan 1.3 specification[M/OL]. 2024[2024-03-16]. <https://registry.khronos.org/vulkan/>.
- [21] 蔡至诚. 混合实时渲染关键技术研究[D]. 杭州: 浙江大学, 2022.